

AT32 PMSM FOC Sensorless Quick Start Guide

Introduction

This application note describes how to quickly drive a sensorless PMSM motor using a motor development board and provides a debugging SOP. ArteryTek offers multiple motor development boards, and this document uses the AT-MOTOR-EVB as an example for demonstration.

Applicable products:

Product series	AT32F402/405 series
	AT32F403A/407 series
	AT32F413 series
	AT32F415 series
	AT32F421 series
	AT32F422/426 series
	AT32F425 series
	AT32F423 series
	AT32F435/437 series
	AT32F45x series
	AT32L021 series
	AT32M412/M416 series

Contents

1	Motor control library algorithms	6
2	Environment requirements	7
2.1	Hardware requirements	7
2.2	Software requirements	9
3	Quick start guide	13
3.1	Modify motor control mode and parameters	14
3.2	Connection settings	18
3.3	Open loop control	19
3.4	Auto identification and tuning of parameters.....	20
3.5	ID tune.....	22
3.6	IQ tune	24
3.7	Motor startup method in sensorless mode.....	26
3.8	Current and angular observer parameter tuning	30
3.9	Sensorless startup + Current loop control	33
3.10	Sensorless startup + speed loop control.....	35
3.11	Flux-Weakening Control (Surface-Mounted Permanent Magnet Synchronous Motor, SPMSM).....	37
3.12	Upwind/downwind startup.....	39
3.13	External voltage control/software control mode	41
4	Revision history.....	43

List of tables

Table 1. Flash memory size and ROM configuration	9
Table 2. Macro definition setting (internal crystal)	14
Table 3. Macro definition setting (EVB version).....	14
Table 4. Macro definition setting (lower-side MOS inverted output).....	14
Table 5. Macro definition setting (vector control mode).....	14
Table 6. Macro definition setting (current sampling method)	14
Table 7. Macro definition setting (two-shunt current sampling method).....	15
Table 9. Macro definition setting (MOSFET's $R_{DS(on)}$)	15
Table 9. Macro definition setting (dual ADCs current sampling)	15
Table 8. Macro definition setting (sensorless control)	15
Table 9. Macro definition setting (D/Q-axis current low-pass filter).....	15
Table 12. Macro definition setting (a/b/c phase current low-pass filter)	15
Table 11. Macro definition setting (upwind/downwind startup).....	16
Table 12. Macro definition setting (voltage sensing detection)	16
Table 10. Macro definition setting (sensorless startup)	16
Table 10. Macro definition setting (sensorless startup)	16
Table 13. Macro definition setting (field weakening control).....	16
Table 14. Macro definition setting (current decouple control).....	16
Table 15. Macro definition setting (winding parameter identification)	17
Table 16. Macro definition setting (Artery motor UI tool)	17
Table 17. Motor parameter definition	17
Table 18. FOC sensorless parameters	28
Table 19. Document revision history.....	43

List of figures

Figure 1. AT-MOTOR-EVB V1.x	7
Figure 2. AT-MOTOR-EVB V2.x	8
Figure 3. AT32F413RCT7 ROM configuration (Keil).....	10
Figure 4. AT32F413RCT7 ROM configuration (AT32IDE)	11
Figure 5. AT32F413RCT7 ROM configuration (IAR).....	12
Figure 6. Connection settings	18
Figure 7. Select open loop control mode.....	19
Figure 8. Open loop control parameters.....	19
Figure 9. Nominal current macro definition	20
Figure 10. Identify winding parameters	20
Figure 11. Motor winding parameters macro definition	21
Figure 12. Nominal voltage.....	21
Figure 13. Auto tune current PI parameters	21
Figure 14. Macro definition of current PI parameters	22
Figure 15. Step current diagram.....	22
Figure 16. Select ID Tune mode.....	22
Figure 17. D-axis current PID and step current parameters	23
Figure 18. Diagram parameter settings (ID tune).....	23
Figure 19. ID tune waveform	23
Figure 20. D-axis current PI control parameter macro definition	24
Figure 21. Select ID Tune mode.....	24
Figure 22. Q-axis current PID and step current parameters	24
Figure 23. Diagram parameter settings (IQ tune).....	25
Figure 24. IQ tune waveform	25
Figure 25. Q-axis current PI control parameter macro definition	25
Figure 26. Sensorless startup definition	26
Figure 27. Constant current/voltage definition.....	27
Figure 27. Open loop startup and initial angle detection.....	28
Figure 28. Align and go startup.....	29
Figure 29. Observer parameters	30
Figure 30. Observer parameters	30
Figure 31. Observer current waveforms	31
Figure 32. BEMF signals	31

Figure 33. BEMF estimation waveforms.....	32
Figure 34. PLL parameters	32
Figure 35. Waveforms for estimated BEMF signals and estimated electrical angle	33
Figure 36. Open loop startup parameters	33
Figure 37. Torque reference	33
Figure 38. Current loop setting	34
Figure 39. Current loop debug waveforms	34
Figure 40. Set PID data, acceleration and deceleration.....	35
Figure 41. Set target speed	35
Figure 42. Diagram parameter settings (speed control).....	35
Figure 43. Waveforms in speed control mode.....	36
Figure 44. FWC PID gains and the maximum negative d-axis current limit	37
Figure 45. Target speed setting (FWC)	37
Figure 46. Diagram parameter settings (speed control).....	38
Figure 47. Waveforms in speed control mode (FWC)	38
Figure 48. Upwind/downwind startup parameter definitions	40
Figure 49. External voltage control – potentiometer	41
Figure 50. Control source definitions (upper: software control, lower: external voltage control).....	41
Figure 51. Speed control mode (external voltage control)	42
Figure 52. Torque control mode (external voltage control).....	42
Figure 53. Set target speed in software control mode	42

1 Motor control library algorithms

This document contains the following motor control algorithms:

Target motor type: Three-phase permanent magnet synchronous motor (BLDC)

Control modes:

- FOC vector control

Three-phase PWM method:

- SVPWM

Phase current sensing methods:

- 3-shunt current sensing
- 2-shunt current sensing
- 1-shunt current sensing and reconstruction method

Initial rotor position estimation mode:

- Three-phase voltage vector mode

Rotor position estimation in FOC driving modes

- BEMF estimation with Luenberger observer

Sensorless FOC sinusoidal control mode:

- Open loop control
- Torque control (current vector control)
- Speed control
- Field weakening control
- Upwind/Downwind startup

Auto tuning functions:

- Motor winding parameters identification
- Current PI control parameters self-tuning

2 Environment requirements

2.1 Hardware requirements

PMSM (BLDC) motor, AT-Link or third-party debugger, and motor control evaluation board (AT-MOTOR-EVB) are required. For details about hardware configurations, please refer to *AT32_LV_Motor_Control_EVB_User_Manual* (UM0011 for EVB_V1.x or UM0014 for EVB_V2.x).

- PMSM (BLDC) motor
- Debugger
- Motor control EVB: AT-MOTOR-EVB V1.x or AT-MOTOR-EVB V2.x

Figure 1. AT-MOTOR-EVB V1.x

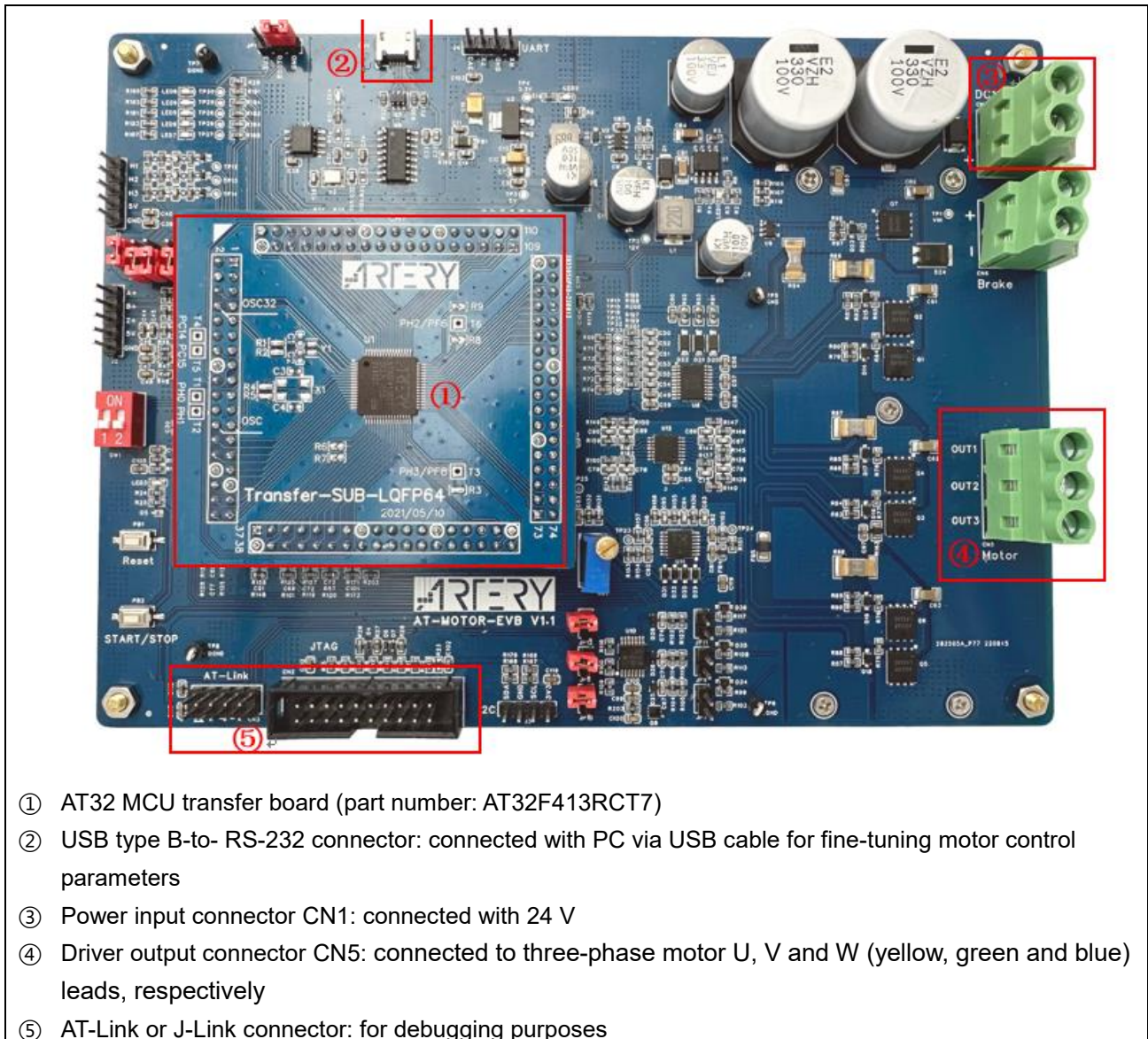
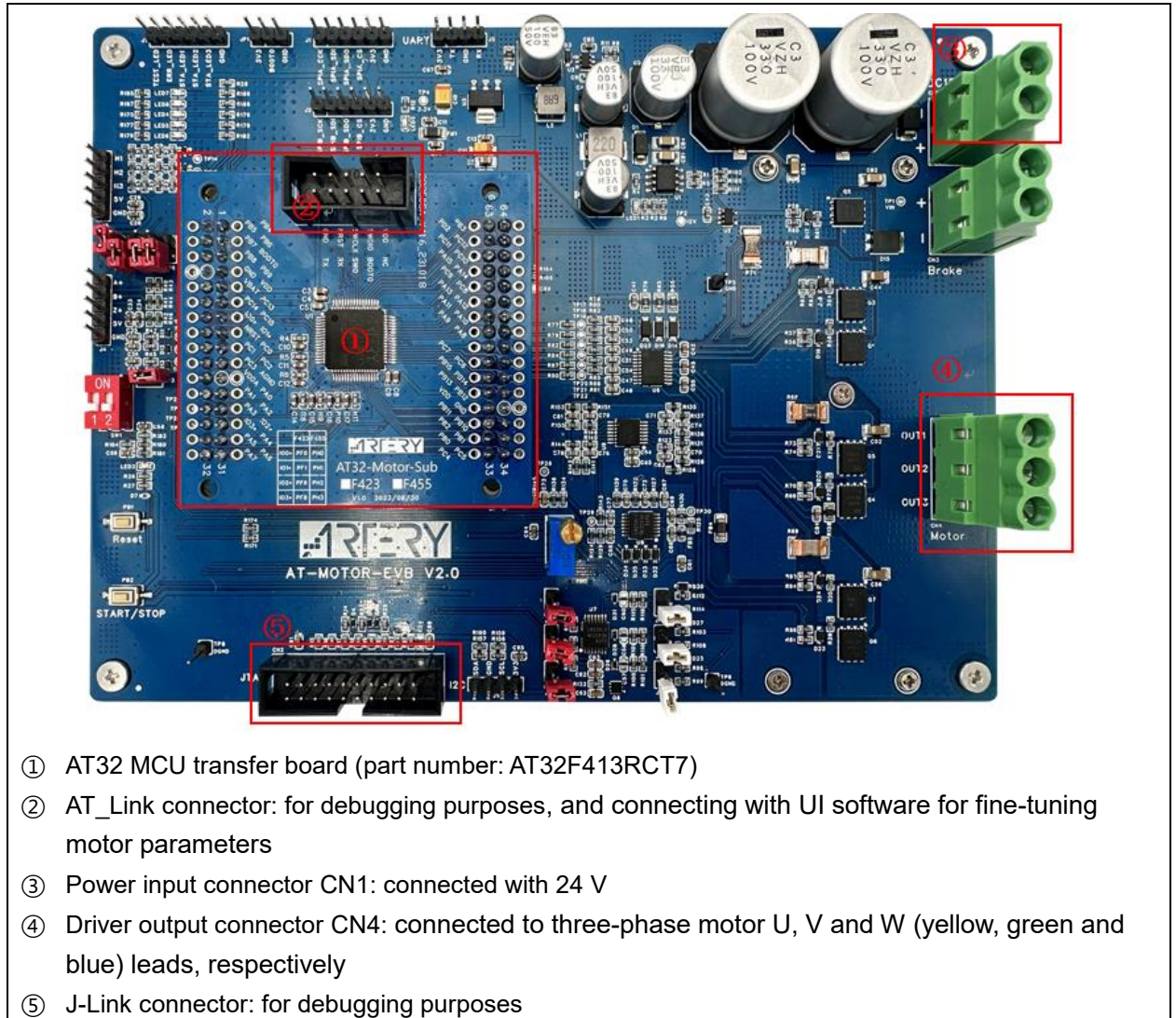


Figure 2. AT-MOTOR-EVB V2.x



2.2 Software requirements

- 1) Taking AT32F413 as an example, open AT32F413_MC_Library_Project\motor_evb_2v0\at32f413\pmsm_foc_sensorless (note [1]).
- 2) Run “ArteryMotorMonitor.exe” (installation not required).
- 3) In Keil, go to **Options - Read/Only Memory Areas** to set ROM according to the Flash memory size of each AT32 MCU (see Table 1). For example: for AT32F413RCT7 with a 256KB Flash memory, its IROM1 start address is at 0x8000000 with a size of 0x3F800, whereas its IROM2 start address is at 0x803F800 with a size of 0x800, as shown in Figure 3. In AT32IDE, modify *ld file according to the Flash memory size of each AT32 MCU (see Figure 4).
In IAR Systems, modify *icf file according to the Flash memory size of each AT32 MCU (see Figure 5).

Table 1. Flash memory size and ROM configuration

Flash size	4032K	1024K	512K	448K	256K
IROM1(address)	0x8000000	0x8000000	0x8000000	0x8000000	0x8000000
IROM1(size)	0x3EF000	0xFF800	0x7F800	0x6F000	0x3F800
IROM2(address)	0x83EF000	0x80FF800	0x807F800	0x0806F000	0x803F800
IROM2(size)	0x1000	0x800	0x800	0x1000	0x800

Flash size	128K	64K	32K	16K
IROM1(address)	0x8000000	0x8000000	0x8000000	0x8000000
IROM1(size)	0x1FC00	0x0FC00	0x07C00	0x03C00
IROM2(address)	0x801FC00	0x800FC00	0x8007C00	0x8003C00
IROM2(size)	0x400	0x400	0x400	0x400

Note [1]: This demo does not support using of Keil complier version 5 and O0 optimization level for compiling purpose simultaneously. Thus either Keil compiler version 6 or O1 optimization level is used for this purpose. As AT32 BSP source code does not support V6.15 compiler, it is recommended to use Keil compiler version 5 and O1 optimization level.

Figure 3. AT32F413RCT7 ROM configuration (Keil)

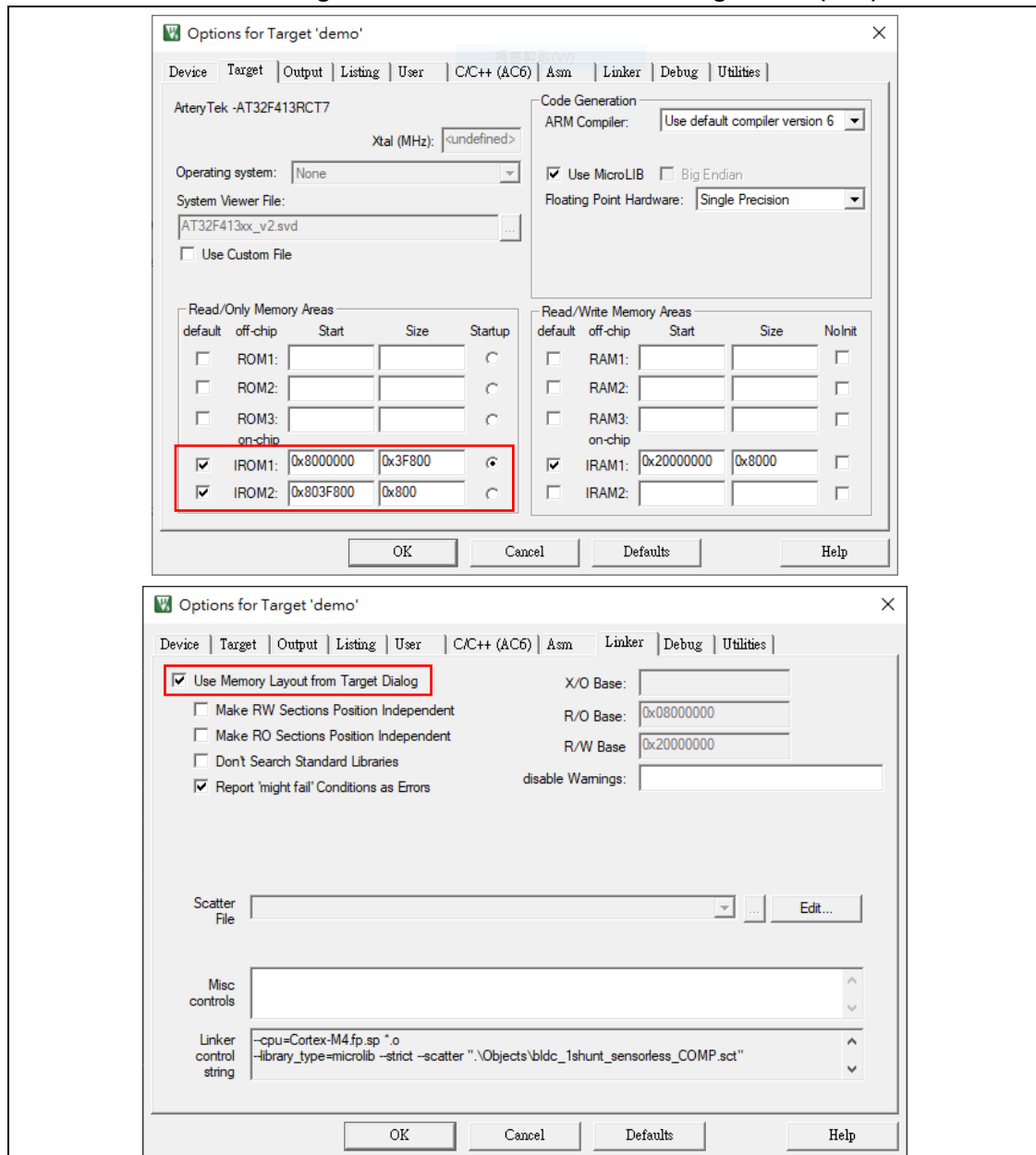


Figure 4. AT32F413RCT7 ROM configuration (AT32IDE)

```

AT32F413xC_FLASH.ld X
25 _estack = 0x20008000; /* end of RAM */
26
27 /* Generate a link error if heap and stack don't fit into RAM */
28 _Min_Heap_Size = 0x200; /* required amount of heap */
29 _Min_Stack_Size = 0x400; /* required amount of stack */
30
31 /* Specify the memory areas */
32 MEMORY
33 {
34     FLASH (rx)      : ORIGIN = 0x08000000, LENGTH = 0x3F800
35     MC_DATA (r)      : ORIGIN = 0x0803F800, LENGTH = 0x800
36     RAM (xrw)       : ORIGIN = 0x20000000, LENGTH = 32K
37 }
38
39 /* Define output sections */
40 SECTIONS
41 {
42     /* The startup code goes first into FLASH */
43     .isr_vector :
44     {
45         . = ALIGN(4);
46         KEEP(*(.isr_vector)) /* Startup code */
47         . = ALIGN(4);
48     } >FLASH
49
50     .mc_data :
51     {
52         . = ALIGN(4);
53         . = ORIGIN(MC_DATA);
54         _MC_VectStoreAddr1 = .;
55         *(.MC_VectStoreAddr1);
56         . = ALIGN(4);
57         . = ORIGIN(MC_DATA) + 800;
58         _MC_VectStoreAddr2 = .;
59         *(.MC_VectStoreAddr2);
60         . = ALIGN(4);
61     } >MC_DATA
62
63     /* The program code and other data goes into FLASH */
64     .text :
65     {
66         . = ALIGN(4);
67         *(.text)           /* .text sections (code) */
68         *(.text*)          /* .text* sections (code) */
69         *(.glue_7)         /* glue arm to thumb code */
70         *(.glue_7t)        /* glue thumb to arm code */
71         *(.eh_frame)
72

```

Figure 5. AT32F413RCT7 ROM configuration (IAR)

```

1  /*###ICF### Section handled by ICF editor, don't touch! *****/
2  /*-Editor annotation file-*/
3  /* IcfEditorFile="$TOOLKIT_DIR$\config\ide\IcfEditor\cortex_v1_0.xml" */
4  /*-Specials-*/
5  define symbol __ICFEDIT_intvec_start__ = 0x08000000;
6  /*-Memory Regions-*/
7  define symbol __ICFEDIT_region_ROM_start__ = 0x08000000;
8  define symbol __ICFEDIT_region_ROM_end__ = 0x0803F800 - 1;
9  define symbol __ICFEDIT_region_ROM1_start__ = 0x0803F800;
10 define symbol __ICFEDIT_region_ROM1_end__ = (__ICFEDIT_region_ROM1_start__ + 800 - 1);
11 define symbol __ICFEDIT_region_ROM2_start__ = (__ICFEDIT_region_ROM1_end__ + 1);
12 define symbol __ICFEDIT_region_ROM2_end__ = 0x0803F800 + 0x800 - 1;
13 define symbol __ICFEDIT_region_RAM_start__ = 0x20000000;
14 define symbol __ICFEDIT_region_RAM_end__ = 0x20007FFF;
15 /*-Sizes-*/
16 define symbol __ICFEDIT_size_cstack__ = 0x400;
17 define symbol __ICFEDIT_size_heap__ = 0x200;
18 /**** End of ICF editor section. ###ICF###*/
19
20 define memory mem with size = 4G;
21 define region ROM_region = mem:[from __ICFEDIT_region_ROM_start__ to __ICFEDIT_region_ROM_end__];
22 define region ROM1_region = mem:[from __ICFEDIT_region_ROM1_start__ to __ICFEDIT_region_ROM1_end__];
23 define region ROM2_region = mem:[from __ICFEDIT_region_ROM2_start__ to __ICFEDIT_region_ROM2_end__];
24 define region RAM_region = mem:[from __ICFEDIT_region_RAM_start__ to __ICFEDIT_region_RAM_end__];
25
26 define block CSTACK with alignment = 8, size = __ICFEDIT_size_cstack__ { };
27 define block HEAP with alignment = 8, size = __ICFEDIT_size_heap__ { };
28
29 initialize by copy { readwrite };
30 do not initialize { section .noinit };
31
32 place at address mem: __ICFEDIT_intvec_start__ { readonly section .intvec };
33
34
35 place in ROM_region { readonly };
36 place in ROM1_region { readonly section .MC_VectStoreAddr1 };
37 place in ROM2_region { readonly section .MC_VectStoreAddr2 };
38 place in RAM_region { readwrite,
39 block CSTACK, block HEAP };

```

3 Quick start guide

Follow the procedures below to fine-tune a PMSM motor by using FOC sensorless control method.

Step 1: Modify motor control drive mode and parameters according to motor parameters and application requirements (see Section 3.1).

Step 2: Connect with UI software (see Section 3.2).

Step 3: Select open loop control mode to check whether motor is running normally (see Section 3.3).

Step 4: Enable winding parameter identification (see Section 3.4).

Step 5: Current control parameters

- Current PI controller parameter auto identification (see Section 3.4)
- ID tune: fine-tuning Kp and Ki values of current loop (see Section 3.5)
- IQ tune: fine-tuning Kp and Ki values of current loop (see Section 3.6)

Step 6: Select a sensorless startup method with constant voltage or constant current activation (see Section 3.7).

Step 7: Tune sensorless observer parameters

- Observer parameters (see Section 3.8)
- PLL observer parameters (see Section 3.8)

***For more information, go to ARTERY official website -> SUPPORT -> Training Center -> AT32 Motor Control Library: AT32F421 FOC Sensorless Control (Part 2)**

Step 8: Sensorless startup + current loop control (see Section 3.9)

Step 9: Sensorless startup + speed loop control (see Section 3.10)

Step 10: Adjust field weakening control parameters as needed (see Section 3.11)

Step 11: If bidirectional wind startup functionality is required, perform parameter adjustments related to tailwind/headwind operation (see Section 3.12)

Step 12: By default, control commands are provided via UI software. To switch to external potentiometer control, adjust the control command source (see Section 0)

3.1 Modify motor control mode and parameters

- The *motor_control_drive_param.h* file allows users to configure the motor drive mode, motor parameters, board parameters and controller parameters.
- Users can configure the desired drive mode according to their hardware and motor configurations, as detailed below.

1. Select to use an internal crystal.

Table 2. Macro definition setting (internal crystal)

Definition	Description
INTERNAL_CLOCK_SOURCE	Use an internal crystal (disabled by default)

2. Select a motor development board (V1 or V2, optional).

Table 3. Macro definition setting (EVB version)

Definition name	Description
AT_MOTOR_EVB_V1	AT-MOTOR-EVB V1.x (include mc_hwio_v1.h file)
AT_MOTOR_EVB_V2	AT-MOTOR-EVB V2.0 (include mc_hwio_v1.h file)

3. Enable or disable lower-side inverted output feature depending on the gate driver used in the user hardware circuit. For example, the U3315 in the AT32_LV_MOTOR_CONTROL_EVB is with this inverted output function, so it is pre-defined to Enable.

Table 4. Macro definition setting (lower-side MOS inverted output)

Definition	Description
GATE_DRIVER_LOW_SIDE_INVERT	Enable lower-side MOS inverted output (enabled by default; comment to disable)

4. This example is a FOC vector control demonstration project, so there is no need to make changes here. This option, however, is reserved for use in code judgment.

Table 5. Macro definition setting (vector control mode)

Definition	Description
FOC_CONTROL	Vector control mode

5. Select a current sampling method from below according to user hardware circuit and application requirements.

Table 6. Macro definition setting (current sampling method)

Definition	Description
THREE_SHUNT	3-shunt current sampling
TWO_SHUNT	2-shunt current sampling
ONE_SHUNT	1-shunt current sampling

6. If there is a need to use 2-shunt current sampling, select one from below according to user hardware circuit and application requirements, if not, this step is not needed.

Table 7. Macro definition setting (two-shunt current sampling method)

Definition	Description
U_V_SHUNT	U & V shunt
V_W_SHUNT	V & W shunt
U_W_SHUNT	U & W shunt

- Based on whether the user's hardware circuit uses the low-side MOSFET's $R_{DS(on)}$ for current sampling.

Table 8. Macro definition setting (MOSFET's $R_{DS(on)}$)

Definition	Description
MOS_RDS_SHUNT	Current Sampling Utilizing Low-Side MOSFET's $R_{DS(on)}$ (disabled by default)

- Based on whether the user's requirements call for using dual ADCs for current sampling (Applicable to MCUs with dual or more ADCs; not suitable for ONE_SHUNT configurations).

Table 9. Macro definition setting (dual ADCs current sampling)

Definition	Description
TWO_ADC_CONVERTERS	Current Sampling with Dual-ADC Synchronization (disabled by default)

- This example is for sensorless project, so the sensorless mode is selected by default, and there is no need to make changes here. This option, however, is reserved for use in code judgment.

Table 10. Macro definition setting (sensorless control)

Definition	Description
SENSORLESS	Sensorless control mode

- Enable D/Q-axis current signal low-pass filtering mode according to actual needs.

Table 11. Macro definition setting (D/Q-axis current low-pass filter)

Definition	Description
CURRENT_LP_FILTER	D/Q-axis current low-pass filter signals (disabled by default for 3 or 2-shunt current sampling, but enabled by default for 1-shunt current sampling)

- Enable a/b/c phase current signal low-pass filter according to user needs.

Table 12. Macro definition setting (a/b/c phase current low-pass filter)

Definition	Description
AC_CURRENT_LP_FILTER	a/b/c phase current low-pass filter (disabled by default)

- Configure bidirectional wind startup based on user requirements. If enabled, back-EMF voltage detection is automatically utilized for wind direction sensing (no additional

activation of VOLT_SENSE function required). For implementation details, refer to Section 3.12

Table 13. Macro definition setting (upwind/downwind startup)

Definition	Description
WIND_SENSE	Upwind/downwind startup (note [2]) (it is disabled by default)

Note [2]: Taking the AT-MOTOR-EVB V1.x development board as an example, components C67, C71, C74, C66, C70, and C72 must be mounted according to the schematic diagram. If the development board is AT-MOTOR-EVB V2.0, components C57, C58, and C60 should be mounted as the schematic. For the AT-MOTOR-EVB V2.1 development board, components C48, C49, and C50 must be mounted, and jumpers JP13, JP14, and JP15 need to be installed according to the schematic.

- The observer's input voltage is configured to optionally use back-EMF voltage detection based on user requirements (no additional activation required if WIND_SENSE is enabled).

Table 14. Macro definition setting (voltage sensing detection)

Definition	Description
VOLT_SENSE	BEMF Voltage sensing in sensorless vector control mode (note [2]) (it is disabled by default)

- Select sensorless constant voltage or constant current startup based on user requirements (enable one option only).

Table 15. Macro definition setting (sensorless startup)

Definition	Description
CONST_CURRENT_START	Constant current startup
CONST_VOLTAGE_START	Constant voltage startup

- Select a sensorless startup method according to actual needs (see Section 3.7).

Table 16. Macro definition setting (sensorless startup)

Definition	Description
OPENLOOP_STARTUP	Open loop startup
ALIGN_AND_GO_STARTUP	Align and go startup
INIT_ANGLE_STARTUP	Initial angle startup

- Enable field weakening control according to actual needs.

Table 17. Macro definition setting (field weakening control)

Definition	Description
FIELD_WEAKENING	Field weakening control (disabled by default)

- Enable current decouple feature during torque (current) control according to user needs.

Table 18. Macro definition setting (current decouple control)

Definition	Description
------------	-------------

CURRENT_DECOUPLE_CTRL	Current decouple control (N/A for AT32L021 series) (disabled by default)
-----------------------	---

18. Enable winding parameter identification. Once enabled, it is used to detect RS and LS of motor windings, and calculate current loop PI control parameters (see Section 3.4).

Table 19. Macro definition setting (winding parameter identification)

Definition	Description
MOTOR_PARAM_IDENTIFY	Motor winding parameter identification (enabled by default; comment to disable)

19. Enable or disable the Artery motor UI tool according to user needs.

Table 20. Macro definition setting (Artery motor UI tool)

Definition	Description
USE_MOTOR_MONITOR	Enable Artery motor UI tool (enabled by default; comment to disable)

Table 21 gives the motor parameters used in the demo, such as number of pole-pairs, encoder parameters and Hall sensor parameters.

Table 21. Motor parameter definition

Definition name	Description
RS_LL	Motor line-to-line winding resistance (unit: Ω)
LS_LL	Motor line-to-line winding inductance (unit: H)
POLE_PAIRS	Number of pole-pairs
KE	Motor KE value (unit: V/rpm) (fill in this value when current decouple control is enabled)
LD_LQ_RETIO	Ld to Lq ratio
NOMINAL_CURRENT	Nominal current (unit: ampere)

3.2 Connection settings

After getting hardware and software ready, connect UI with the motor control evaluation board as follows. For details, refer to *AN0063_AT32 Motor Monitor Application Note*.

Step 1:

Connect the motor, AT-Link/J-Link and power supply to the motor control evaluation board, and connect USART interface to PC via an USB cable.

Step 2:

Use MDK to compile demo project code, and download code to the motor control evaluation board via J-Link or AT-Link.

Step 3:

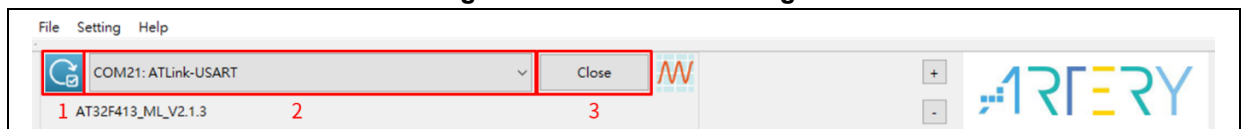
Run ArteryMotorMonitor.exe.

Go to File -> Open Project, and select ArteryMotorMonitor_V2.1.x.atmcx-> Open.

Step 4:

Click the update icon (1.) of Serial Port and select the corresponding serial port (2.); then click Open(3.) to enable real-time communication, as shown in Figure 6.

Figure 6. Connection settings

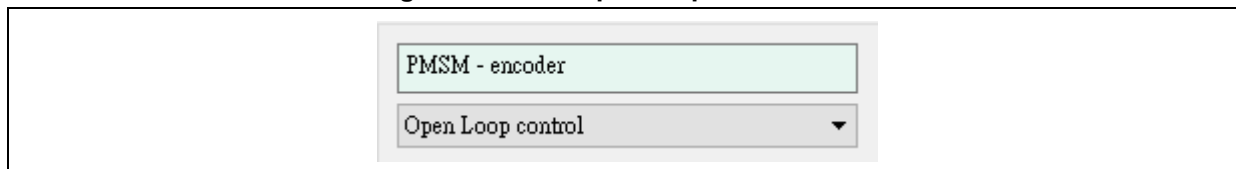


3.3 Open loop control

Using open-loop voltage control, this mode allows motor rotation without position sensors. It serves to confirm the motor's operational status and direction. The open-loop voltage and angular increment are adjusted incrementally from zero, based on the motor's speed and phase current.

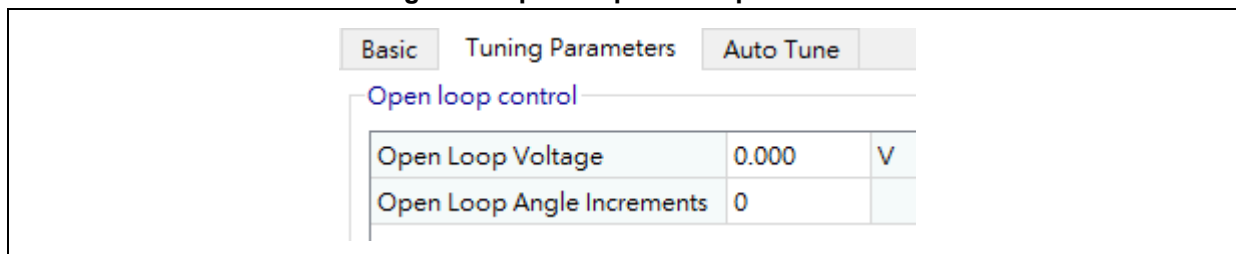
Step 1: Select "Open loop control" through the drop-down menu.

Figure 7. Select open loop control mode



Step 2: Go to "Tuning Parameters" to set "Open Loop Voltage" and "Open Loop Angle Increments". Click on "Start Motor" and monitor the current until the motor starts to run properly (set the open loop voltage value appropriately to prevent overheating damage to the motor).

Figure 8. Open loop control parameters



Tips:

Initially, set "Open Loop Voltage" value to 0.5~2V and set "Open Loop Angle Increments" value to 1~10, and monitor whether the motor starts to run. If not, check whether the 3-phase cables are correctly connected.

3.4 Auto identification and tuning of parameters

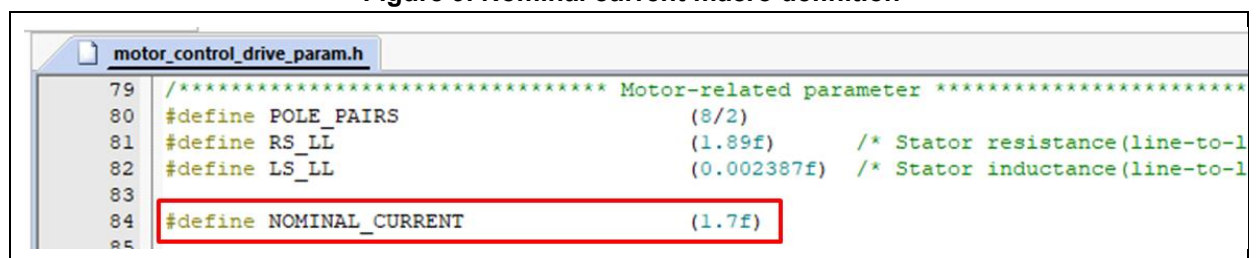
Automatic parameter identification is divided into two parts, namely automatic identification of motor winding parameters and self-tuning of current PI controller parameters. Sensorless angle estimation algorithms require accurate motor parameters to achieve precise angular position measurements.

Auto identification of motor winding parameters are not required if there are RS_LL and LS_LL parameters. Users can directly modify macro definitions of RS_LL and LS_LL in *motor_control_drive_param.h* file.

Auto identification of motor winding parameters:

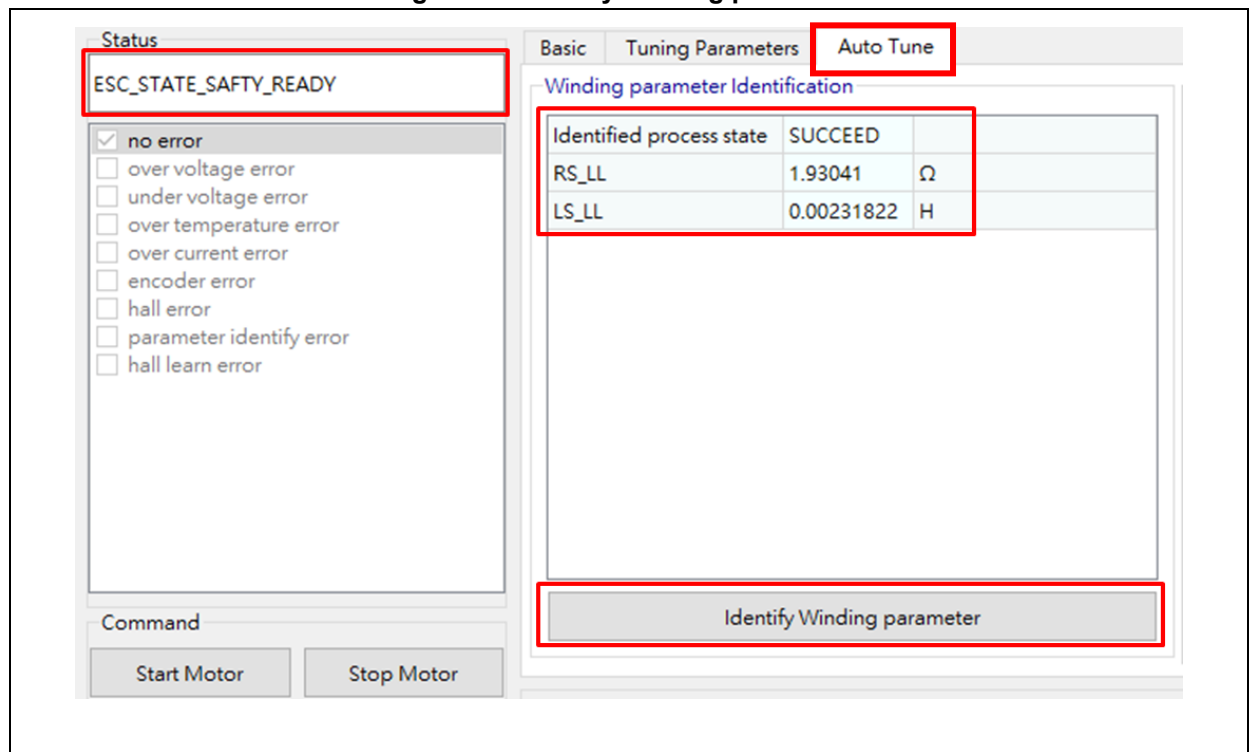
Step 1: Write the macro definition NOMINAL_CURRENT into the *motor_control_drive_param.h* file, and then re-compile and re-program code, as shown in Figure 9.

Figure 9. Nominal current macro definition



Step 2: Go to UI --> Auto Tune, and click on “Identify Winding parameter” when the state machine is in “safety ready” status. This step is to enable winding parameters identification feature, as shown in Figure 10.

Figure 10. Identify winding parameters



Step 3: Check and confirm “SUCCEED” of “Identified process state”; get the RS_LL and LS_LL values and fill them into the corresponding macro definitions in *motor_control_drive_param.h* file, and then re-compile and re-program code, as shown in Figure 11.

Figure 11. Motor winding parameters macro definition

```

motor_control_drive_param.h
89
90 /***** Motor-related parameter *****/
91 /* Motor parameters */
92 #define RS_LL (1.89) /* Stator resistance, ohm */
93 #define LS_LL (0.002387) /* Stator inductance, H */

```

Current PI controller parameter auto tuning:

Step 1: Fill in the macro definition of input DC voltage (VDC_RATED) into the *motor_control_drive_param.h* file, and then re-compile and re-program code, as shown in Figure 12.

Figure 12. Nominal voltage

```

motor_control_drive_param.h
100
101 /***** Drive-related parameter *****/
102 /* basic */
103 #define VDC_RATED (24.0f)
104 #define V_SENSE_GAIN (10/(3.9f+180+10)) // 0.05157
105 #define ADC_REFERENCE_VOLT (3.3f)
106 #define ADC_DIGITAL_SCALE_12BITS (4095)

```

Step 2: Go to “Auto Tune” page, click on “Auto-tune Current PI parameter” in “safety ready” status to perform auto tuning of current PI controller parameters. After auto tuning is completed, Q-axis and D-axis current PI controller parameters are automatically updated, as shown in Figure 13.

Figure 13. Auto tune current PI parameters

Auto-tune PI parameter of Current controller

Torque KP	7717
Torque KI	6110
Torque KP DIV	512
Torque KI DIV	8192

Auto-tune Current PI parameter

Step 3: Write these values obtained in Figure 13 into the corresponding macro definition (*motor_control_drive_param.h*), then recompile and re-program code, as shown in Figure 14.

Figure 14. Macro definition of current PI parameters

```

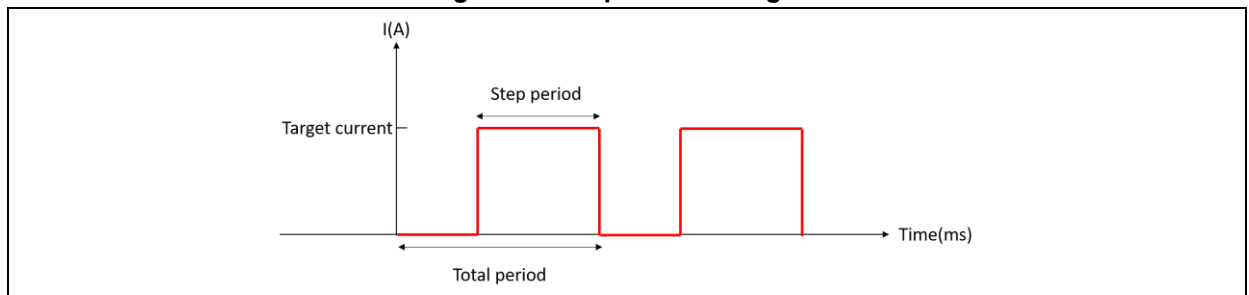
motor_control_drive_param.h
244  /* Id/Iq pid parameter */
245  #define PID_ID_KP_DEFAULT      (7717)
246  #define PID_ID_KI_DEFAULT      (6110)
247  #define PID_ID_KP_GAIN_DIV     (512)
248  #define PID_ID_KP_GAIN_DIV_LOG (LOG2(PID_ID_KP_GAIN_DIV))
249  #define PID_ID_KI_GAIN_DIV     (8192)
250  #define PID_ID_KI_GAIN_DIV_LOG (LOG2(PID_ID_KI_GAIN_DIV))
251
252  #define PID_IQ_KP_DEFAULT      (7717)
253  #define PID_IQ_KI_DEFAULT      (6110)
254  #define PID_IQ_KP_GAIN_DIV     (512)
255  #define PID_IQ_KP_GAIN_DIV_LOG (LOG2(PID_IQ_KP_GAIN_DIV))
256  #define PID_IQ_KI_GAIN_DIV     (8192)
257  #define PID_IQ_KI_GAIN_DIV_LOG (LOG2(PID_IQ_KI_GAIN_DIV))

```

3.5 ID tune

A step current is generated in IQ Tune mode, as shown in Figure 15. Parameters related to the step current are adjustable. The step current is generated to help check the current response after adjusting PID parameters of D axis current.

Figure 15. Step current diagram



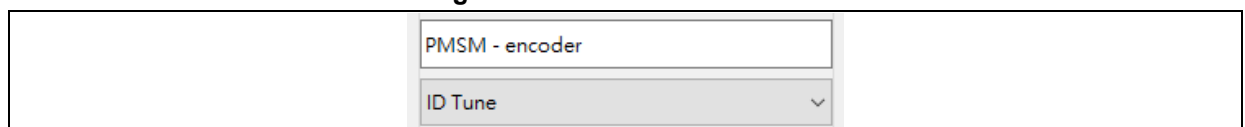
Note: It is recommended to tune D-axis current and then Q-axis current, and to set KP and KI to 0 for D-axis and Q-axis current to avoid unmatched default values affecting current parameter adjustment.

Note: It is recommended to set D-axis current command to 5-10% of the rated current and then slowly increase the D-axis current to the target value to align the rotor with D-axis, so that to protect IPM or MOSFET components from being burnt out by current surges.

Follow the steps bellow:

Step 1: Select "ID tune" through the drop-down menu.

Figure 16. Select ID Tune mode



Step 2: Go to "Tuning Parameters" window to set PID parameters and unit step current parameters. Parameters obtained by current PI controller parameters auto tuning can be used for initial adjustment, and users can adjust these parameters as needed to get the desired current response, as shown in Figure 17.

Figure 17. D-axis current PID and step current parameters

Basic			Tuning Parameters			Auto Tune		
Unit step config						ID Current control		
Current Tune target current	0.999	A	Flux KP	7500				
Current Tune total period	100	ms	Flux KI	6000				
Current Tune step period	2	ms	Flux KP DIV	512				
			Flux KI DIV	8192				

Step 3: Click on “Start Motor”.

Step 4: Set “Flux reference(Id)” and “Flux measured(Id)” in “Diagram parameter setting”, and then click on “Save”.

Figure 18. Diagram parameter settings (ID tune)

Diagram parameter setting

Flux reference (Id)

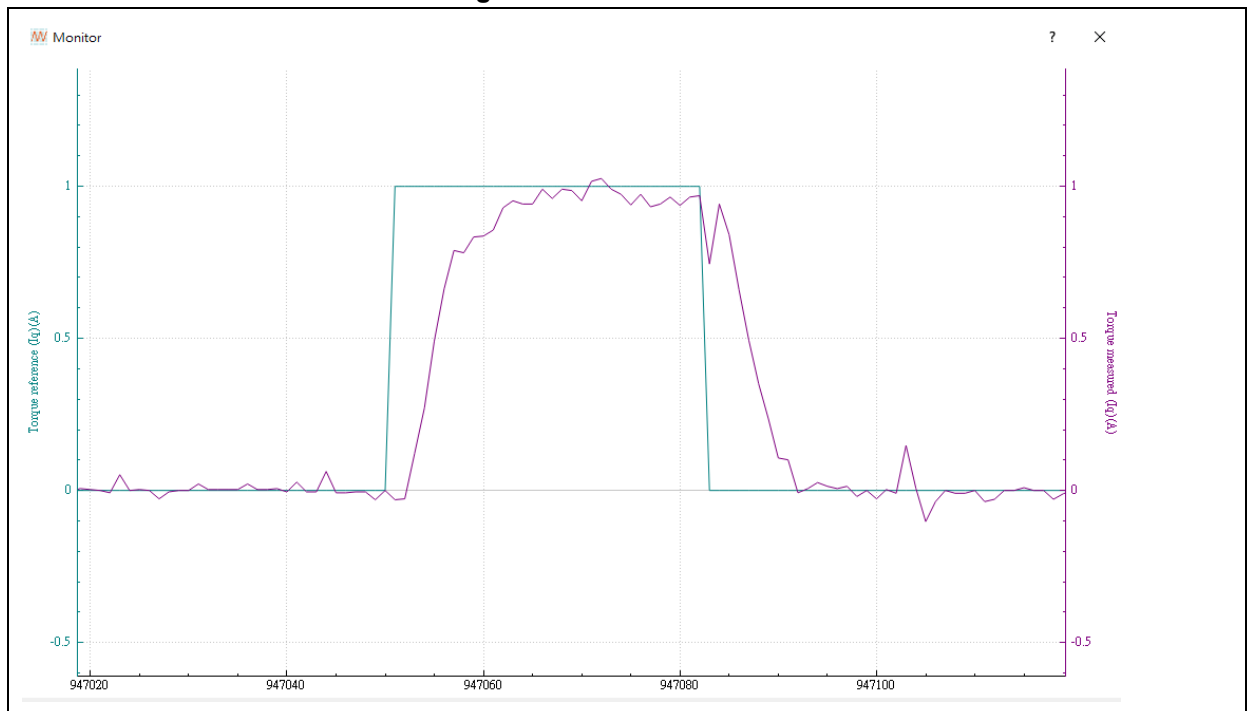
Flux measured (Id)

Save

Step 5: Click  icon to generate waveforms.

Step 6: Check whether the current response is as expected, as shown in Figure 19. If not, click to stop the motor and repeat Step 2~Step 6.

Figure 19. ID tune waveform



STEP-7: After ID tuning is completed, fill parameters into the macro definition (motor_control_drive_param.h), and then re-compile and re-program code, as shown in Figure 20.

Figure 20. D-axis current PI control parameter macro definition

motor_control_drive_param.h		
245	#define	PID_ID_KP_DEFAULT (7500)
246	#define	PID_ID_KI_DEFAULT (6000)
247	#define	PID_ID_KP_GAIN_DIV (512)
248	#define	PID_ID_KP_GAIN_DIV_LOG (LOG2 (PID_ID_KP_GAIN_DIV))
249	#define	PID_ID_KI_GAIN_DIV (8192)
250	#define	PID_ID_KI_GAIN_DIV_LOG (LOG2 (PID_ID_KI_GAIN_DIV))

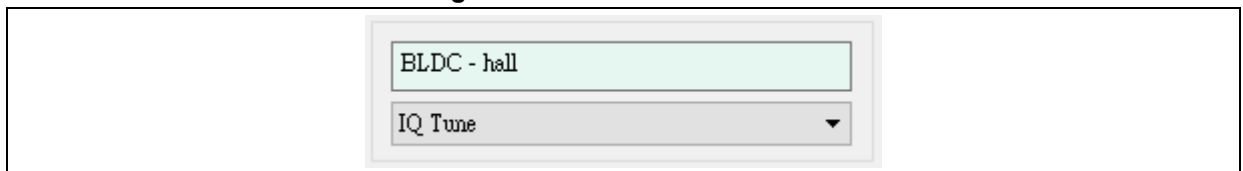
3.6 IQ tune

A step current is generated in IQ Tune mode, as shown in Figure 15. The Step current-related parameters are adjustable. The Step current is used to view Q-axis current response waveforms after IQ PID parameters tuning operation.

Follow the steps bellow:

Step 1: Select "IQ tune" through the drop-down menu.

Figure 21. Select ID Tune mode



Step 2: Go to "Tuning Parameters" window to set PID parameters and unit step current parameters. Parameters obtained by current PI controller parameters auto tuning can be used for initial adjustment, and users can adjust these parameters as needed to get the desired current response, as shown in Figure 22.

Figure 22. Q-axis current PID and step current parameters

Basic

Tuning Parameters

Auto Tune

Unit step config

Current Tune target current	0.999	A
Current Tune total period	100	ms
Current Tune step period	2	ms

IQ Current control

Torque KP	7500	
Torque KI	6000	
Torque KP DIV	512	
Torque KI DIV	8192	

Step 3: Click on "Start Motor".

Step 4: Set "Torque reference(Iq)" and "Torque measured(Iq)" in "Diagram parameter setting", and then click on "Save".

Figure 23. Diagram parameter settings (IQ tune)

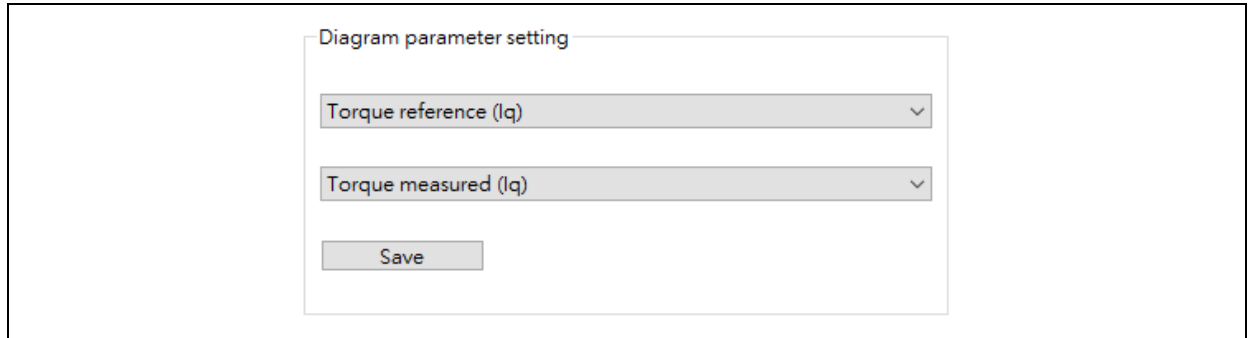


Diagram parameter setting

Torque reference (Iq) ▼

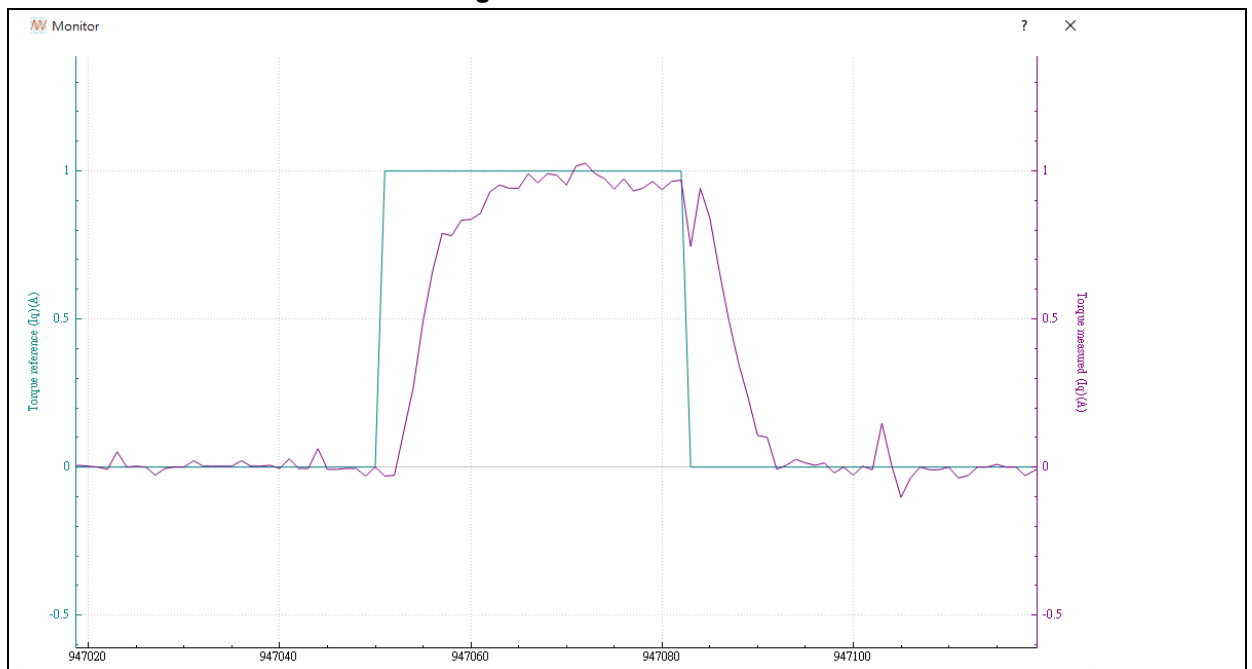
Torque measured (Iq) ▼

Save

Step 5: Click  icon to generate waveforms. For more information, please refer to *AN0063 AT32 Motor Monitor Application Note*.

Step 6: Check whether the current response is as expected, as shown in Figure 24. If not, click to stop the motor and repeat Step 2~Step 6.

Figure 24. IQ tune waveform



STEP-7: After IQ tuning is completed, fill parameters into the macro definition (motor_control_drive_param.h), and then re-compile and re-program code, as shown in Figure 25.

Figure 25. Q-axis current PI control parameter macro definition

```

motor_control_drive_param.h
252 #define PID_IQ_KP_DEFAULT      (7500)
253 #define PID_IQ_KI_DEFAULT      (6000)
254 #define PID_IQ_KP_GAIN_DIV     (512)
255 #define PID_IQ_KP_GAIN_DIV_LOG (LOG2 (PID_IQ_KP_GAIN_DIV))
256 #define PID_IQ_KI_GAIN_DIV     (8192)
257 #define PID_IQ_KI_GAIN_DIV_LOG (LOG2 (PID_IQ_KI_GAIN_DIV))

```

3.7 Motor startup method in sensorless mode

Since sensorless FOC control method with rotor position by estimate back-EMF signals, the absence of back-EMF at motor standstill necessitates alternative methods to determine the initial rotor angle. This enables application of the correct current vector control for proper motor startup. This demo offers three methods for motor startup in sensorless mode, details as follows:

1. Open loop startup

Definition: OPENLOOP_STARTUP

Strength: easy to use

Weakness: may experience reverse phenomenon, with many parameters to tune

Applicable motor type: suitable for light-load motors

2. Align and Go startup

Definition: ALIGN_AND_GO_STARTUP

Strength: Easy-to-use with few parameters to tune, suitable for most motors

Weakness: may experience reverse phenomenon

Applicable motor type: suitable for most types of motors

3. Initial rotor position startup

Definition: INIT_ANGLE_STARTUP

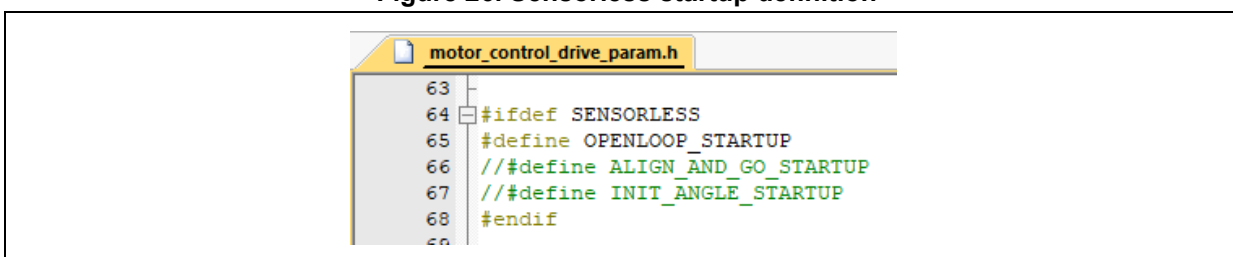
Strength: No reverse

Weakness: not suitable for motors with low magnetic saturation (high inductance/small rated current) or coreless motor

Applicable motor type: motors prone to magnetic saturation and equipped with iron-core structures

Configuration: The three sensorless startup methods can be directly selected by modifying program definitions in the motor_control_drive_param.h header file. This example uses the open-loop startup method, as illustrated in Figure 26.

Figure 26. Sensorless startup definition



Furthermore, two starting control methods are available: constant voltage startup and constant current startup:

1. Constant Voltage Startup

Definition: CONST_VOLTAGE_START

Strength: High success rate for light-load or stable-load startups; facilitates smooth transition to closed-loop control.

Weakness: Unstable starting torque.

2. Constant Current Startup

Definition: CONST_CURRENT_START

Strength: Stable starting torque (suitable for applications requiring high torque or with varying loads).

Weakness: Prone to startup failure in high-voltage systems or environments susceptible to current sampling interference.

The startup method can be configured by modifying the corresponding macro definitions in motor_control_drive_param.h. This example uses the constant current startup method, as shown in Figure 27.

Figure 27. Constant current/voltage definition

```
/* choose const current/voltage start-up */  
#ifndef SENSORLESS  
#define CONST_CURRENT_START  
//#define CONST_VOLTAGE_START  
#endif
```

For the aforementioned three methods, the following parameters (corresponding to Figure 28 and Figure 29 respectively) in Table 22 must be adjusted.

Table 22. FOC sensorless parameters

Sensorless startup method	Definition name	Description
OPENLOOP_STARTUP	STARTUP_CURRENT	Open-loop control current command (unit: ampere)
	STARTUP_VOLTAGE	Open-loop control voltage command (unit: ampere)
	STARTUP_MAX_SPD	Maximum open-loop speed before closed-loop transition (unit: rpm)
	STARTUP_OL_SLOPE	Open loop startup acceleration (unit: rpm/s)
ALIGN_AND_GO_STARTUP	STARTUP_CURRENT	Alignment / Startup current command (unit: ampere)
	STARTUP_VOLTAGE	Alignment / Startup voltage command (unit: ampere)
	STARTUP_ALIGN_TIME	Pre-Startup rotor alignment time (unit: ms)
	STARTUP_START_TIME	Post-Alignment closed-loop transition time (unit: ms)
INIT_ANGLE_STARTUP	DETECT_PULSE_WIDTH	Initial rotor angle detection pulse width (unit: us)
	STARTUP_CURRENT	Open-loop control current command (unit: ampere)
	STARTUP_VOLTAGE	Open-loop control voltage command (unit: ampere)
	STARTUP_MAX_SPD	Maximum open-loop speed before closed-loop transition (unit: rpm)
	STARTUP_OL_SLOPE	Open-loop startup acceleration (unit: rpm/s)

Figure 28. Open loop startup and initial angle detection

Start up CONST_CURRENT_START		
Startup Start Current	0.499	A
Startup Max. Speed	400	rpm
Startup Open Loop Slope	800	rpm/s
Start up CONST_VOLTAGE_START		
Startup Start Voltage	1.000	V
Startup Max. Speed	400	rpm
Startup Open Loop Slope	800	rpm/s

Figure 29. Align and go startup

Start up

CONST_CURRENT_START

Startup Start Current	0.499	A
Startup Align Time	1000	ms
Satrtup Start Time	10	ms

Start up

CONST_VOLTAGE_START

Startup Start Voltage	1.000	V
Startup Align Time	1000	ms
Startup Start Time	10	ms

3.8 Current and angular observer parameter tuning

After setting current controller, it is now ready to configure sensorless control parameters. The first step is to adjust current observer parameters in open loop control mode based on the procedures below:

***For more information, go to ARTERY's official website -> SUPPORT -> Training Center -> AT32 Motor Control Library: AT32F421 FOC Sensorless Control (Part 2)**

Step 1: Select Open Loop Control.

Step 2: Set Open Loop Voltage and Open Loop Angle Increments, and press Start Motor (See Section 3.3).

Step 3: Select I_{α} and Observer I_{α} , and click on "Save".

Figure 30. Observer parameters

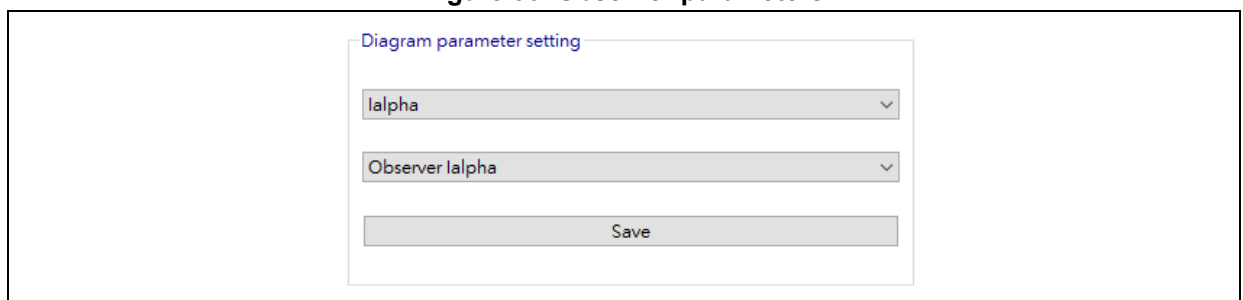


Diagram parameter setting

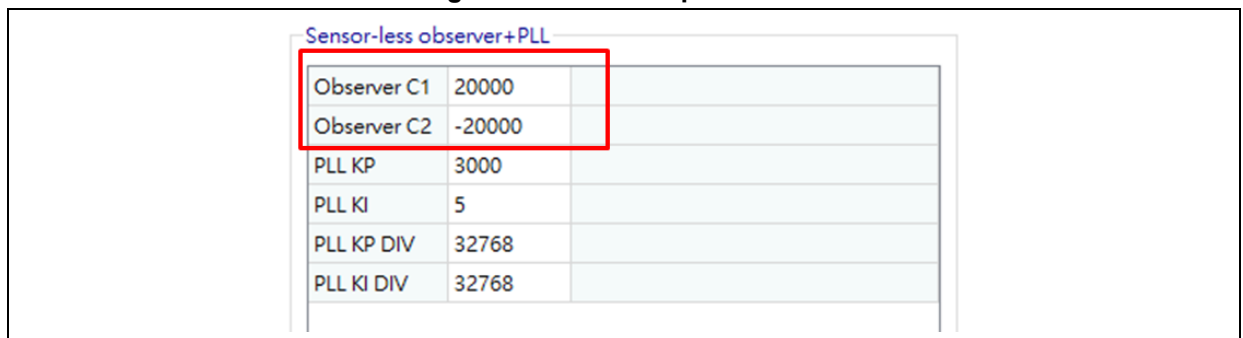
I_{α}

Observer I_{α}

Save

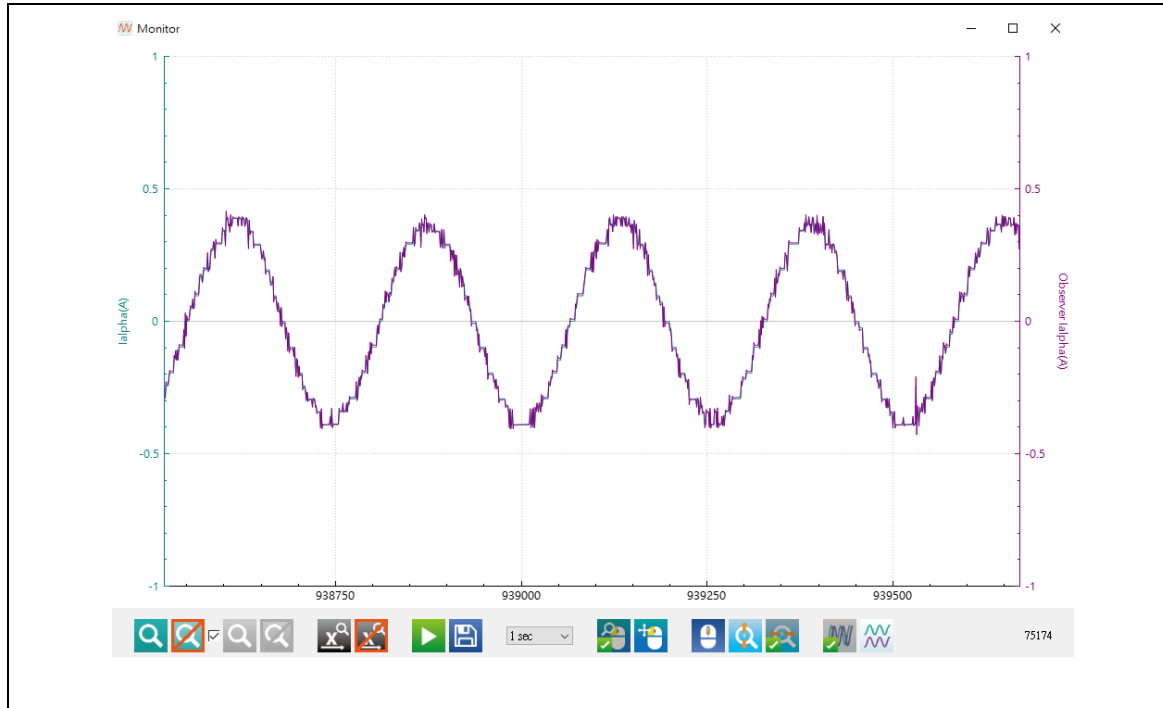
Step 4: Fine-tune Observer C1 and Observer C2 in Figure 31 so that the waveforms are nearly the same for I_{α} and Observer I_{α} in Figure 32.

Figure 31. Observer parameters



Observer C1	20000	
Observer C2	-20000	
PLL KP	3000	
PLL KI	5	
PLL KP DIV	32768	
PLL KI DIV	32768	

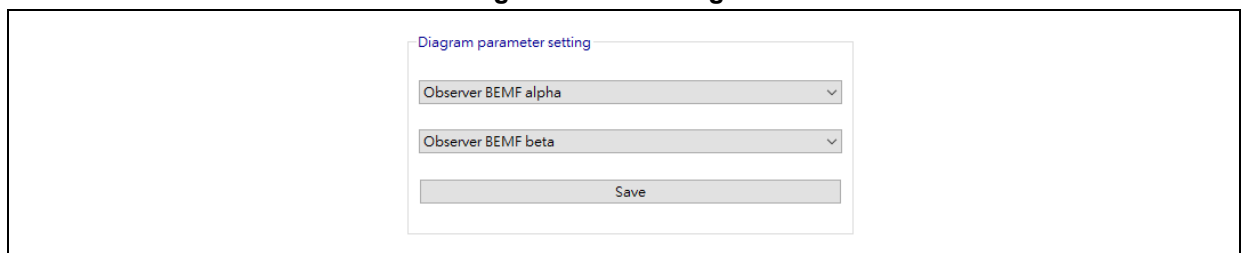
Figure 32. Observer current waveforms



Follow the procedures below to adjust PLL observer:

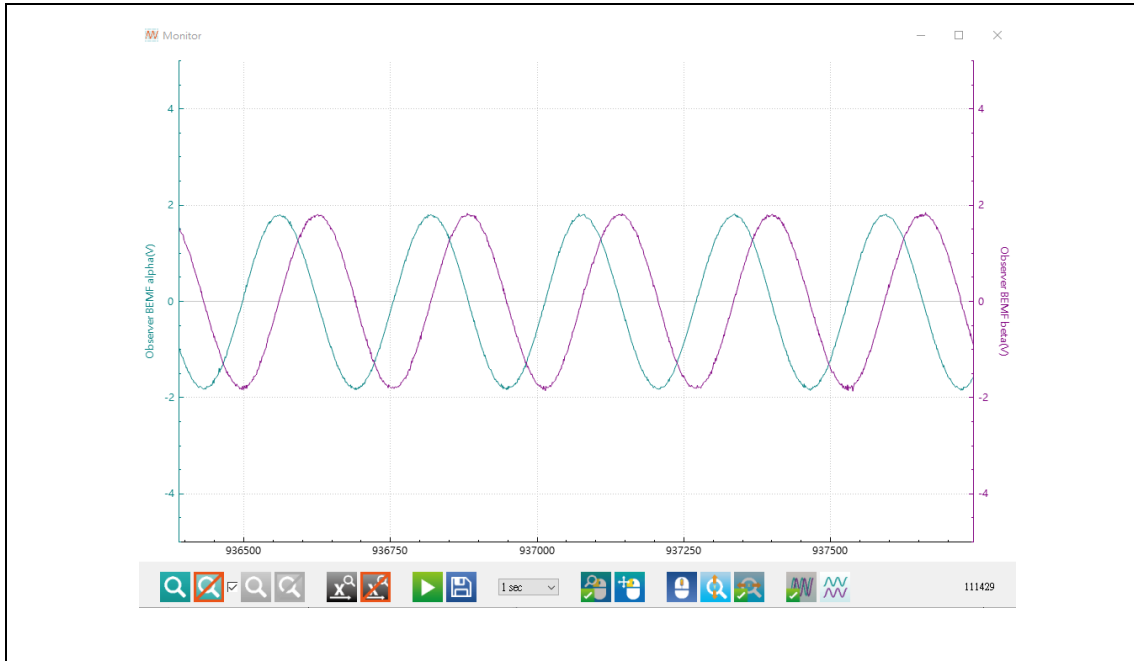
Step 5: Select Observer BEMF alpha and Observer BEMF beta, and click on “Save”.

Figure 33. BEMF signals



Step 6: Verify whether the observer-estimated Back-EMF signals exhibit orthogonal sinusoidal waveforms as shown in Figure 34.

Figure 34. BEMF estimation waveforms



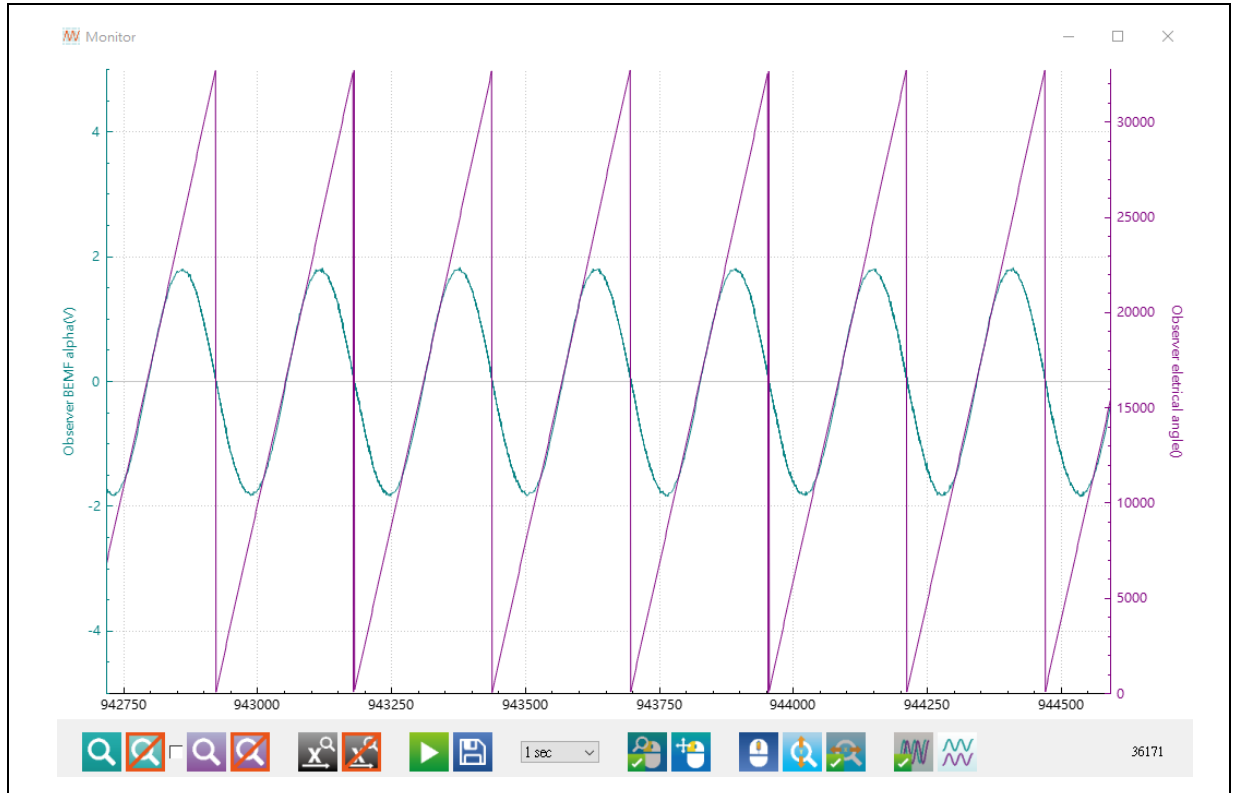
Step 7: Select Observer BEMF alpha and Observer electrical angle, and click on “Save”.

Step 8: As shown in Figure 35, fine-tune PLL values to verify whether the observer-estimated Back-EMF signals and the electrical angle phase align with the reference waveforms depicted in Figure 36.

Figure 35. PLL parameters

Sensor-less observer+PLL	
Observer C1	20000
Observer C2	-20000
PLL KP	3000
PLL KI	5
PLL KP DIV	32768
PLL KI DIV	32768

Figure 36. Waveforms for estimated BEMF signals and estimated electrical angle



3.9 Sensorless startup + Current loop control

In this mode, the motor first performs open-loop startup (using sensorless open-loop startup with constant current activation as an example) and transitions to current closed-loop control upon reaching the preset maximum open-loop speed to achieve torque control. Detailed operational steps are as follows:

Step 1: Select “Torque Control” through the drop-down menu.

Step 2: Set “Startup Start Current”, “Startup Max. Speed” and “Startup Open Loop Slope”, as shown in Figure 37.

Figure 37. Open loop startup parameters

Start up		
Startup Start Current	0.499	A
Startup Max. Speed	400	rpm
Startup Open Loop Slope	800	rpm/s

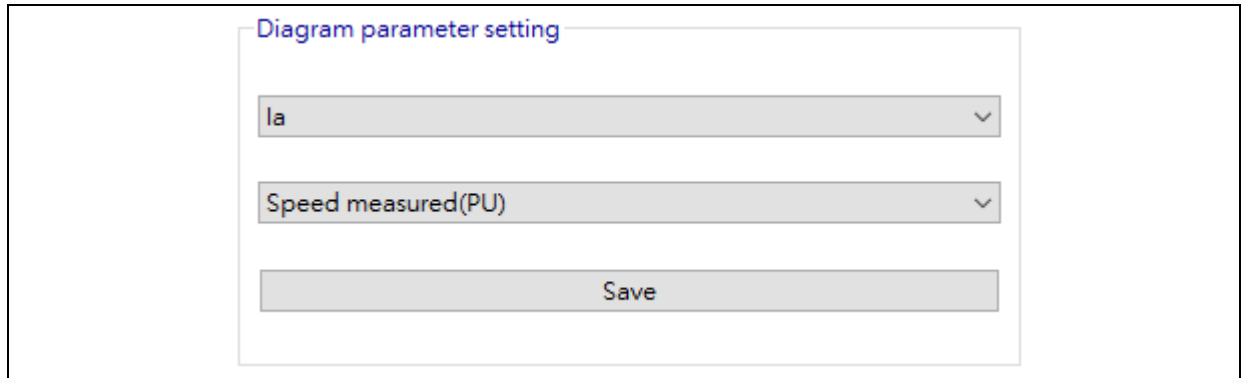
Step 3: Select software control as control source and set Torque reference.

Figure 38. Torque reference

Torque control		
Torque reference (Iq)	0.200	A
Flux reference (Id)	0.000	A

Step 4: Select Ia and Speed measured(PU) and click on “Save”.

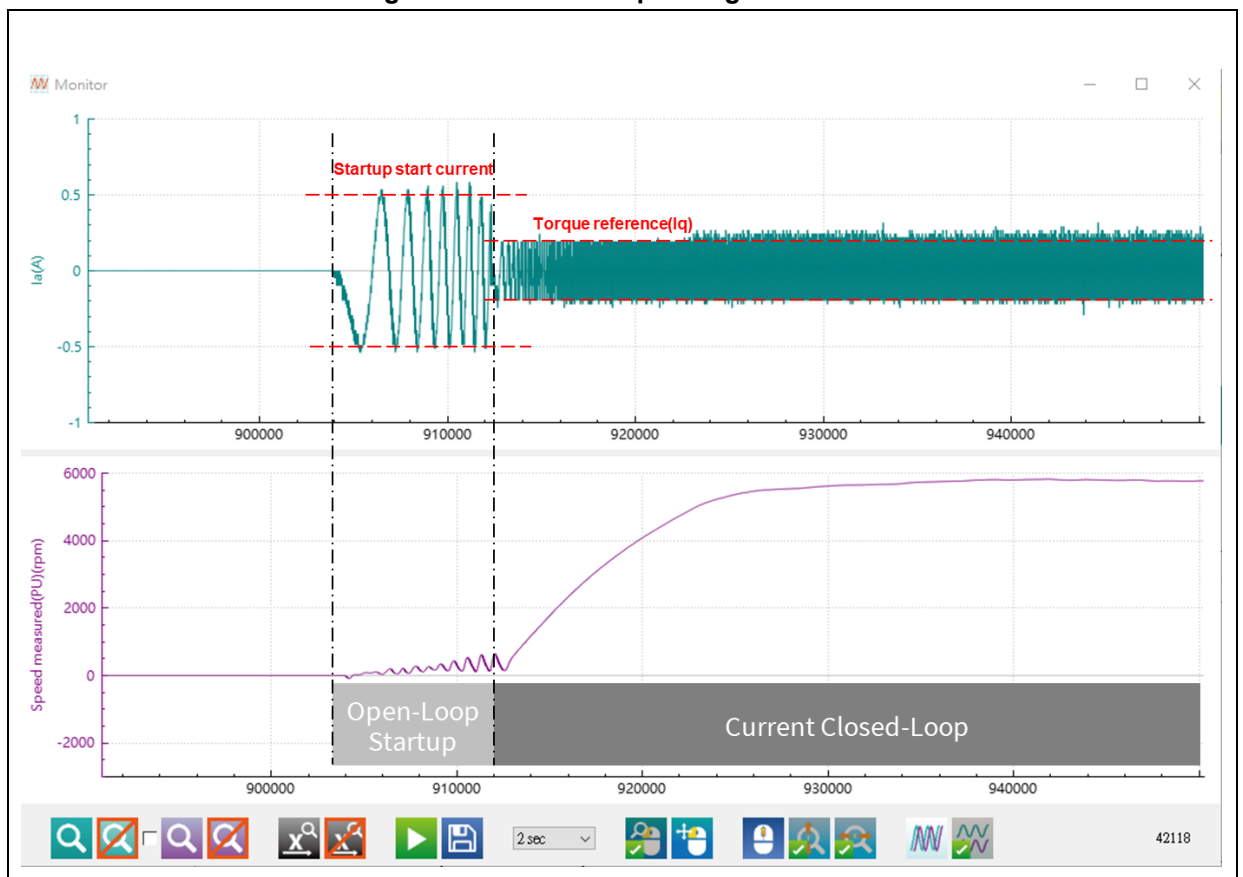
Figure 39. Current loop setting



Step 5: Click on “” to open a waveform window.

Step 6: Click on “Start Motor” to check if the current response is as expected, as shown in Figure 40.

Figure 40. Current loop debug waveforms



3.10 Sensorless startup + speed loop control

In this mode, the motor initiates operation through an open-loop startup (using sensorless open-loop startup with constant current activation as an example) and transitions to speed closed-loop control upon reaching the preset maximum open-loop speed for precise speed regulation. Adjustable parameters include speed loop PID gains, acceleration/deceleration rates, and real-time waveform visualization for response validation. Detailed operational steps are as follows:

Step 1: Select “Speed Control” through the drop-down menu.

Step 2: Set Startup Start Current, Startup Max. Speed and Startup Open Loop Slope, as shown in Figure 37.

Step 3: Set speed PID parameters and acceleration and deceleration.

Figure 41. Set PID data, acceleration and deceleration

Speed control		
Speed KP	1000	
Speed KI	4	
Speed KP DIV	1024	
Speed KI DIV	1024	
Speed acceleration	8	rpm/ms
Speed deceleration	8	rpm/ms

Step 4: Select “software control” in “Control source” and set the target speed (Speed reference).

Figure 42. Set target speed

Target speed		
Speed reference	0	rpm

Step 5: Select Speed reference(PU) and Speed measured(PU), and click on “Save”.

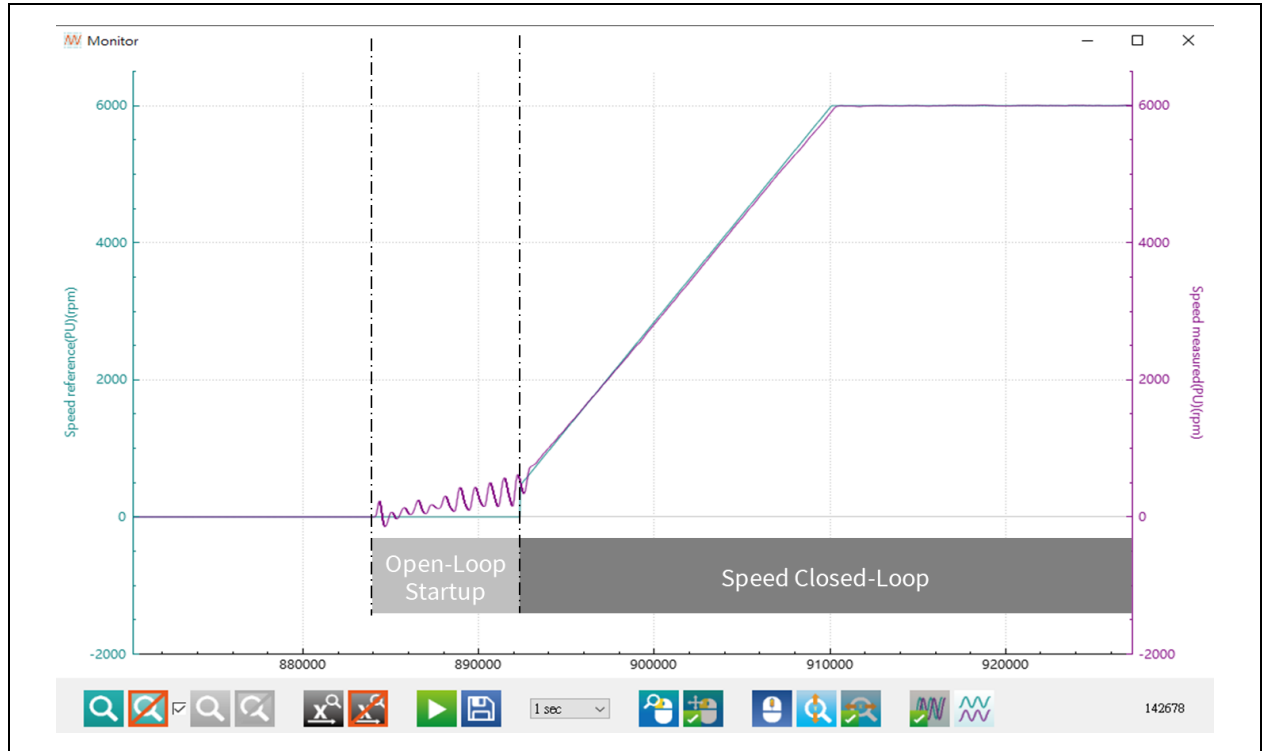
Figure 43. Diagram parameter settings (speed control)

Diagram parameter setting	
Speed reference(PU)	▼
Speed measured(PU)	▼
Save	

Step 6: Click  icon to generate waveforms.

Step 7: Click on “Start motor” to check whether the speed response is as expected, as shown in Figure 44. If not, click to stop the motor and repeat Step 3~Step 7.

Figure 44. Waveforms in speed control mode



Note: The maximum displayed values for speed waveforms (Speed Reference (PU) and Speed Measured (PU)) are defined as $1.25 \times \text{MAX_SPEED_RPM}$, with the minimum being $-1.25 \times \text{MAX_SPEED_RPM}$. If values exceed these limits, the waveform display will clip (overflow), though the actual numerical values remain unaffected. It is recommended to set MAX_SPEED_RPM to the motor's rated speed to avoid display anomalies.

3.11 Flux-Weakening Control (Surface-Mounted Permanent Magnet Synchronous Motor, SPMSM)

This mode supports Surface-Mounted Permanent Magnet Synchronous Motors (SPMSM). During speed-loop control, it injects negative d-axis current commands to weaken the permanent magnet flux linkage, thereby extending the motor's operational speed range. Adjustable parameters include the flux-weakening control (FWC) PID gains and the maximum negative d-axis current limit. After tuning, the dynamic response can be visualized via waveform plotting. Detailed implementation steps are as follows:

Step 1: Select "Speed Control" through the drop-down menu.

Step 2. Navigate to the Tuning Parameters page, configure the maximum d-axis (flux-weakening) current command and adjust the flux-weakening PID parameters.

Note: Configure the maximum d-axis (flux-weakening) current command according to the motor's rated specifications. Exceeding this limit may cause irreversible demagnetization of permanent magnets or thermal damage due to excessive copper/core losses.

Figure 45. FWC PID gains and the maximum negative d-axis current limit

-Flux weakening tuning		
Max. Flux weakening Current	1.4	A
Flux weakening KP	300	
Flux weakening KI	100	
Flux weakening KP DIV	2048	
Flux weakening KI DIV	32768	

Step 3. Set Control Source to Software Control and input the Speed Reference value. For example: 8000rpm

Note: The flux-weakening control (FWC) is triggered only when the motor speed exceeds the base speed (i.e., the maximum sustainable speed under rated flux linkage).

Figure 46. Target speed setting (FWC)

Target speed	
Speed reference	0 rpm

Step 4: Select Speed reference(PU) and Speed measured(PU), and click on "Save".

Figure 47. Diagram parameter settings (speed control)

Diagram parameter setting

Speed reference(PU)
▼

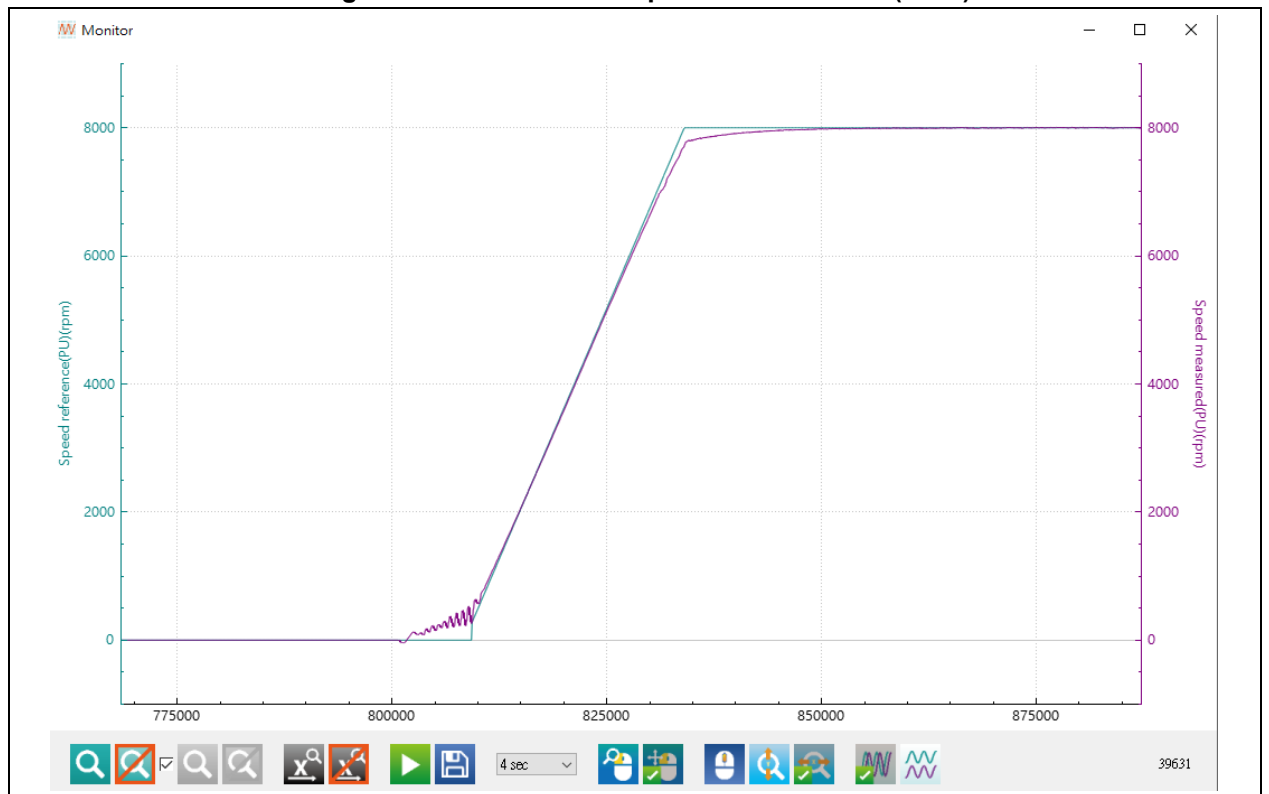
Speed measured(PU)
▼

Save

Step 5: Click  icon to generate waveforms.

Step 6: Click on “Start motor” to check whether the speed response is as expected, as shown in Figure 48. If not, click to stop the motor and repeat Step 2~Step 6.

Figure 48. Waveforms in speed control mode (FWC)



3.12 Upwind/downwind startup

This function is commonly applied in fan-driven systems (e.g., high-speed hair dryers, blowers, electric fans, server cooling fans). These motors typically utilize sensorless vector control, where back-EMF is captured from phase voltage measurements before startup, and an observer estimates rotor speed and direction. Based on the detected speed and rotation direction, adaptive startup strategies are implemented under different conditions.

Three Startup Scenarios and Strategies

Scenario 1: Forward Wind with Sufficient Speed

After the forward/reverse wind detection period (`WIND_DETECT_TIME`), if the motor is detected under forward wind (tailwind) with a speed exceeding the minimum closed-loop transition speed (`TAILWIND_SPEED`), the system directly switches to closed-loop control to drive the motor to the target speed or torque.

Scenario 2: Insufficient Speed or Zero Speed

If, after `WIND_DETECT_TIME`, the motor is under forward wind but below `TAILWIND_SPEED`, or under reverse wind (headwind) with a speed greater than `-TAILWIND_SPEED`, or at zero speed, the system initiates a sensorless startup method (refer to Section 3.7).

Scenario 3: Reverse Wind with Significant Backspin

If, after `WIND_DETECT_TIME`, the motor is under reverse wind with a speed below `-TAILWIND_SPEED`, the system triggers three-phase lower MOSFET arm short-circuit braking. After the reverse wind braking duration (`HEADWIND_BRAKE_TIME`), it proceeds with the sensorless startup method (refer to Section 3.7).

Three critical macros are predefined for customization:

WIND_DETECT_TIME:

Description: Wind direction and speed detection window.

Unit: Milliseconds (ms).

Usage: Duration for sampling back-EMF to determine initial rotor state before startup.

TAILWIND_SPEED:

Description: Minimum speed threshold for direct closed-loop transition under forward wind (tailwind).

Unit: Revolutions per minute (RPM).

Usage: If detected speed exceeds this value in forward wind, the system bypasses startup and enters closed-loop control.

HEADWIND_BRAKE_TIME:

Description: Braking duration applied during reverse wind (headwind) with significant backspin.

Unit: Milliseconds (ms).

Usage: Time for three-phase lower MOSFET short-circuit braking to decelerate the motor before restarting.

These parameters are defined in `motor_control_drive_param.h` and can be modified directly in the code. Refer to Figure 49 in the documentation for their exact location.

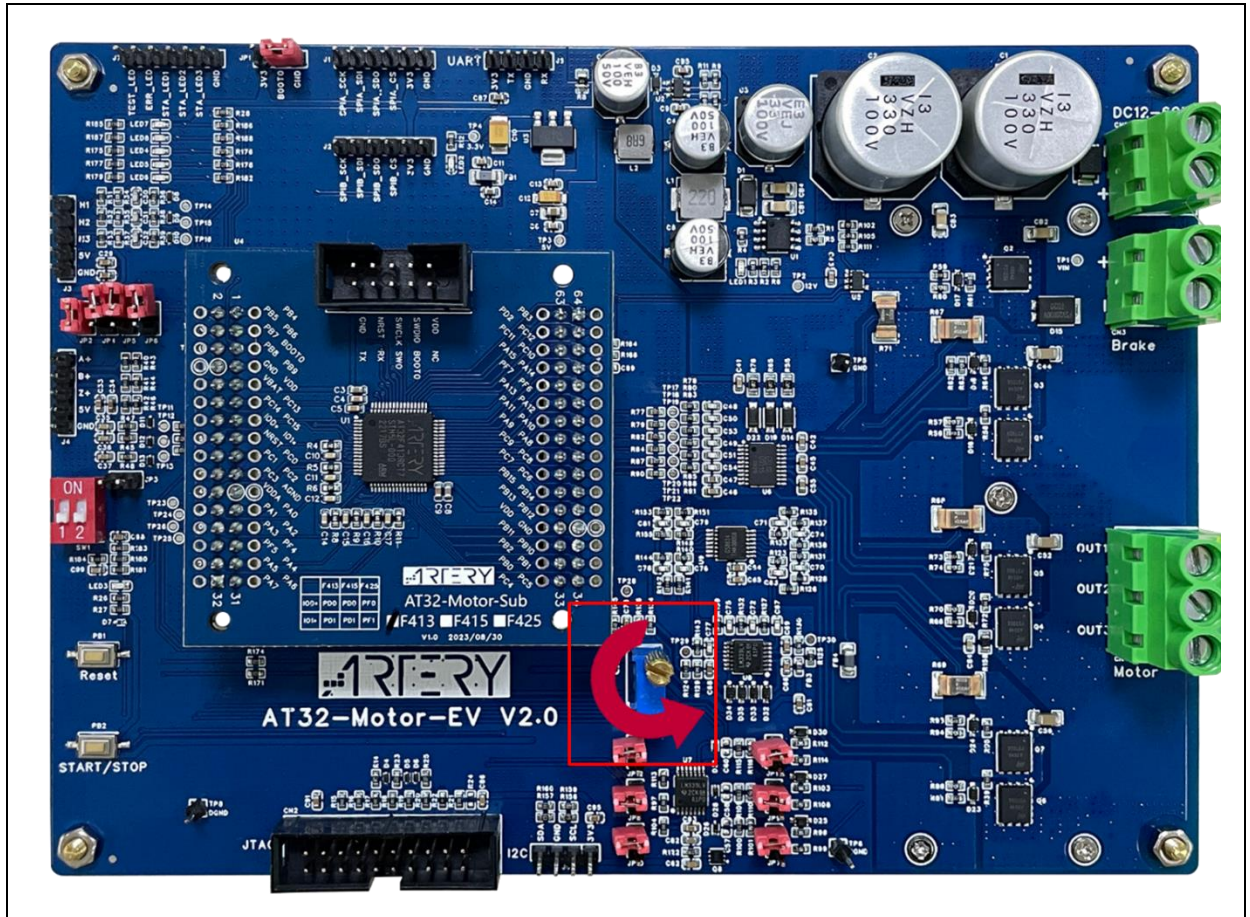
Figure 49. Upwind/downwind startup parameter definitions

```
/* headwind and tailwind detection */  
#define WIND_DETECT_TIME          (200)      /* ms */  
#define TAILWIND_SPEED            (100)      /* rpm */  
#define HEADWIND_BRAKE_TIME      (750)      /* ms */
```

3.13 External voltage control/software control mode

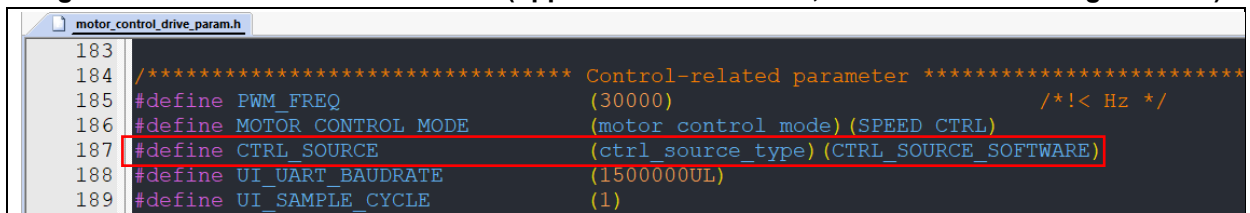
This demo project support two control sources, i.e., external voltage control and software control. The external voltage control mode adjusts speed or torque through external voltage. Users can adjust the potentiometer (as shown in the red box below) to control the command value. The voltage increases when the potentiometer is turned **counterclockwise**. The software control mode inputs speed or torque command value for control purpose. In this demo project, software control mode is set by default.

Figure 50. External voltage control – potentiometer



Users can select the control source by modifying the corresponding definition (CTRL_SOURCE in *motor_control_drive_param.h* file, as shown in Figure 51) or through PC software (see Figure 52). Control parameters are different for different control modes, such as the target speed (as shown in Figure 52) in speed control mode, or torque reference (as shown in Figure 53) in torque control mode (as shown in Figure 52).

Figure 51. Control source definitions (upper: software control, lower: external voltage control)




```

183
184 /***** Control-related parameter *****/
185 #define PWM_FREQ (30000) /*!< Hz */
186 #define MOTOR_CONTROL_MODE (motor_control_mode)(SPEED_CTRL)
187 #define CTRL_SOURCE (ctrl_source_type)(CTRL_SOURCE_EXTERNAL)
188 #define UI_UART_BAUDRATE (1500000UL)
189 #define UI_SAMPLE_CYCLE (1)
    
```

1) Select “External voltage control” as a control source

Select “external voltage control” in “Control source”. In this mode, it is possible to adjust speed or torque value by setting external voltage values. The “Target speed” and “Torque reference” area will generate the control speed or torque calculated based on the current voltage.

Note: The “Target speed” and “Torque reference” cannot be modified in “external voltage control” mode.

Figure 52. Speed control mode (external voltage control)

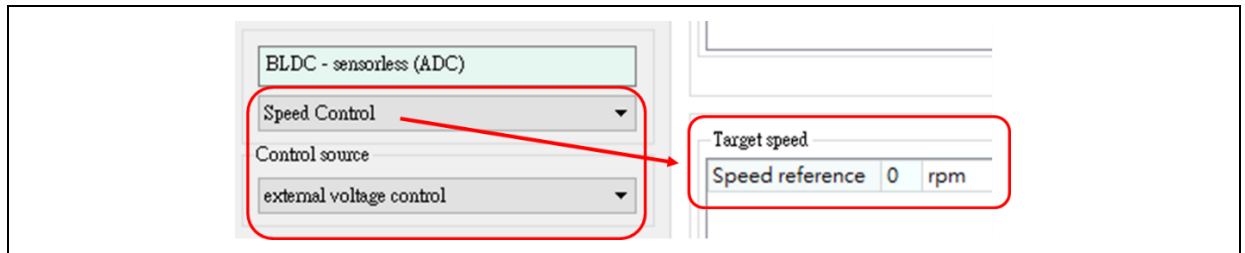
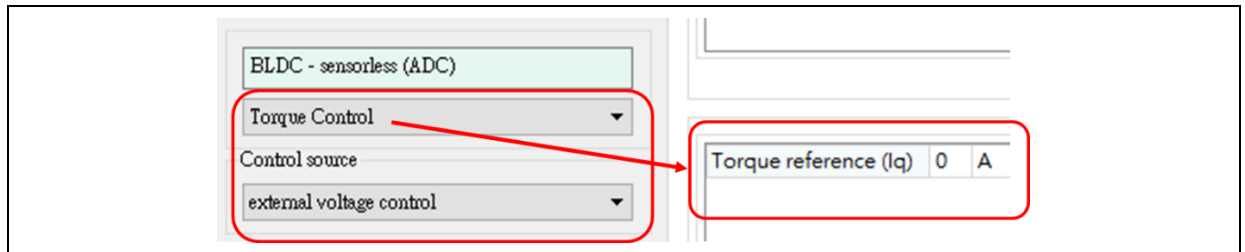


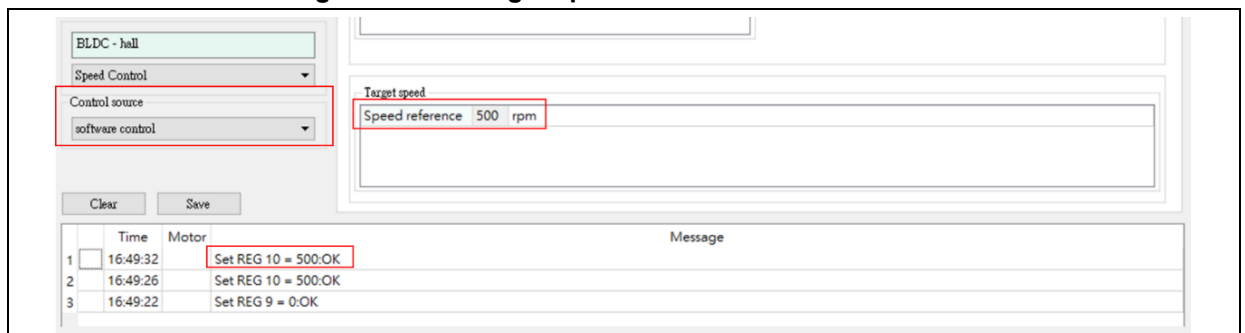
Figure 53. Torque control mode (external voltage control)



2) Select “Software control” as a control source

The software control mode is selected by default, as shown in Figure 54. In this mode, the user can adjust the motor speed and torque by changing the target speed/torque reference. The “message” area will show whether the changing operation is successful or not.

Figure 54. Set target speed in software control mode



4 Revision history

Table 23. Document revision history

Date	Version	Revision note
2024.05.24	2.0.0	Initial release.
2024.12.27	2.0.1	1. Added AT32F435/437 series and AT32L021 series support; 2. Added description of IAR environment and *.icf file modification for Flash configuration; 3. Updated definitions, code and some descriptions.
2025.04.11	2.1.2	1. Added AT32F425 series support; 2. Added sensorless upwind/downwind startup macro definitions and functional description
2025.10.09	2.1.3	Added constant voltage startup method, expanded supported models, corrected flash configuration tables, and updated parameters.

IMPORTANT NOTICE – PLEASE READ CAREFULLY

Purchasers are solely responsible for the selection and use of ARTERY's products and services; ARTERY assumes no liability for purchasers' selection or use of the products and the relevant services.

No license, express or implied, to any intellectual property right is granted by ARTERY herein regardless of the existence of any previous representation in any forms. If any part of this document involves third party's products or services, it does NOT imply that ARTERY authorizes the use of the third party's products or services, or permits any of the intellectual property, or guarantees any uses of the third party's products or services or intellectual property in any way.

Except as provided in ARTERY's terms and conditions of sale for such products, ARTERY disclaims any express or implied warranty, relating to use and/or sale of the products, including but not restricted to liability or warranties relating to merchantability, fitness for a particular purpose (based on the corresponding legal situation in any unjudicial districts), or infringement of any patent, copyright, or other intellectual property right.

ARTERY's products are not designed for the following purposes, and thus not intended for the following uses: (A) Applications that have specific requirements on safety, for example: life-support applications, active implant devices, or systems that have specific requirements on product function safety; (B) Aviation applications; (C) Aerospace applications or environment; (D) Weapons, and/or (E) Other applications that may cause injuries, deaths or property damages. Since ARTERY products are not intended for the above-mentioned purposes, if purchasers apply ARTERY products to these purposes, purchasers are solely responsible for any consequences or risks caused, even if any written notice is sent to ARTERY by purchasers; in addition, purchasers are solely responsible for the compliance with all statutory and regulatory requirements regarding these uses.

Any inconsistency of the sold ARTERY products with the statement and/or technical features specification described in this document will immediately cause the invalidity of any warranty granted by ARTERY products or services stated in this document by ARTERY, and ARTERY disclaims any responsibility in any form.

© 2025 ARTERY Technology – All Rights Reserved